Universität Konstanz

Introduction to Computing for the Social Sciences

Session 6 – Recursion

David Garcia

Winter Semester 2026-2027

Course Outline

- Session 1: Introduction
- Session 2: Information Coding
- Session 3: Data Structures
- Session 4: Software
- Session 5: Algorithms
- Session 6: Recursion
- Session 7: Sorting Algorithms
- Session 8: Time Complexity
- Session 9: Theoretical Computer Science
- Session 10: Machine Learning
- Session 11: Databases
- Session 12: Parallel Programming
- Session 13: Social Science Applications
- Session 14: Exam Review

How many people report to her?

Universität Konstanz

MOTIVATING EXAMPLE

An organizational chart has a chief executive, division heads, department heads, team leads and staff, and **nobody knows how deep it goes**, because it differs by division. You are asked a simple question: how many people, in total, sit below any given manager?

- A loop over levels needs to know **how many levels** to go through.
- You don’t. And the answer changes next month.

How do you write a procedure that works without knowing in advance how deep the structure is?

The comment thread with no bottom

Universität Konstanz

MOTIVATING EXAMPLE

A discussion thread has replies, replies to replies, and replies to those. To count the posts, render the page, or measure how deep an argument went, something has to walk the whole structure, and the structure is **defined in terms of itself**:

A comment is

a post, plus a list of **comments**.

When the *data* is defined in terms of itself, what should the *procedure* look like?

The algorithm that is 2 300 years old

Universität Konstanz

MOTIVATING EXAMPLE

Euclid's *Elements*, Book VII, around 300 BC, gives a procedure for the greatest common divisor: **replace the larger number by its remainder against the smaller, and repeat.** It is still the algorithm your computer uses today.

gcd(270, 192)

gcd(270, 192) → gcd(192, 78) → gcd(78, 36) → gcd(36, 6) → gcd(6, 0) = **6**

What makes this procedure *stop*? And what happens to a procedure that calls itself and forgets to?

Euclid, *Elements*, Book VII, Propositions 1–2

Recursion example



Simpleland

Recursion example

A person (P) is a citizen of Simpleland if it satisfies any of these conditions:

- P was born in Simpleland
- either of P's parents are citizens of Simpleland



Simpleland

Recursion example

A person (P) is a citizen of Simpleland if it satisfies any of these conditions:

- P was born in Simpleland
- either of P's parents are citizens of Simpleland

Is P a citizen of Simpleland?



Simpleland



Recursion example

Universität Konstanz

A person (P) is a citizen of Simpleland if it satisfies any of these conditions:

- P was born in Simpleland
- either of P's parents are citizens of Simpleland

Is P a citizen of Simpleland?



Simpleland

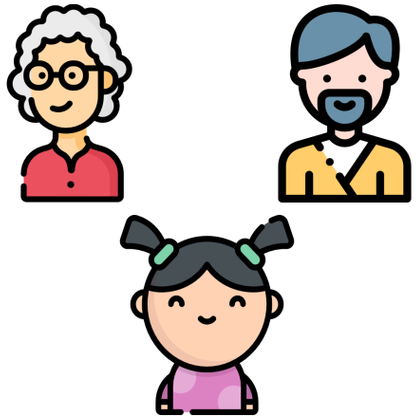


Recursion example

A person (P) is a citizen of Simpleland if it satisfies any of these conditions:

- P was born in Simpleland
- either of P's parents are citizens of Simpleland

Is P a citizen of Simpleland?



Simpleland

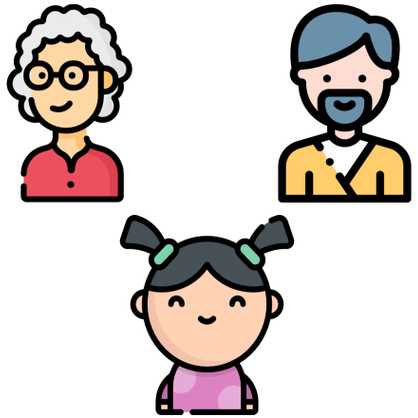


Recursion example

A person (P) is a citizen of Simpleland if it satisfies any of these conditions:

- P was born in Simpleland
- either of P's parents are citizens of Simpleland

Is P a citizen of Simpleland?



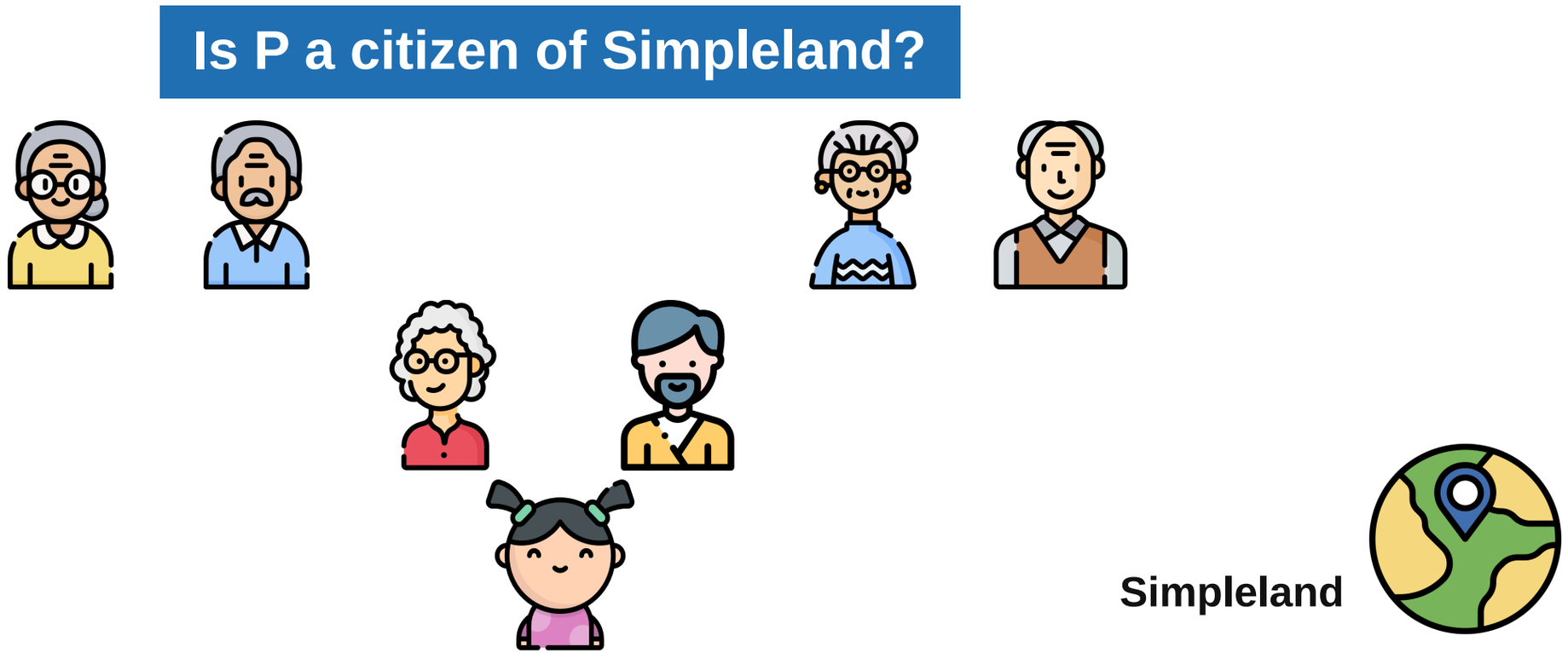
Simpleland



Recursion example

A person (P) is a citizen of Simpleland if it satisfies any of these conditions:

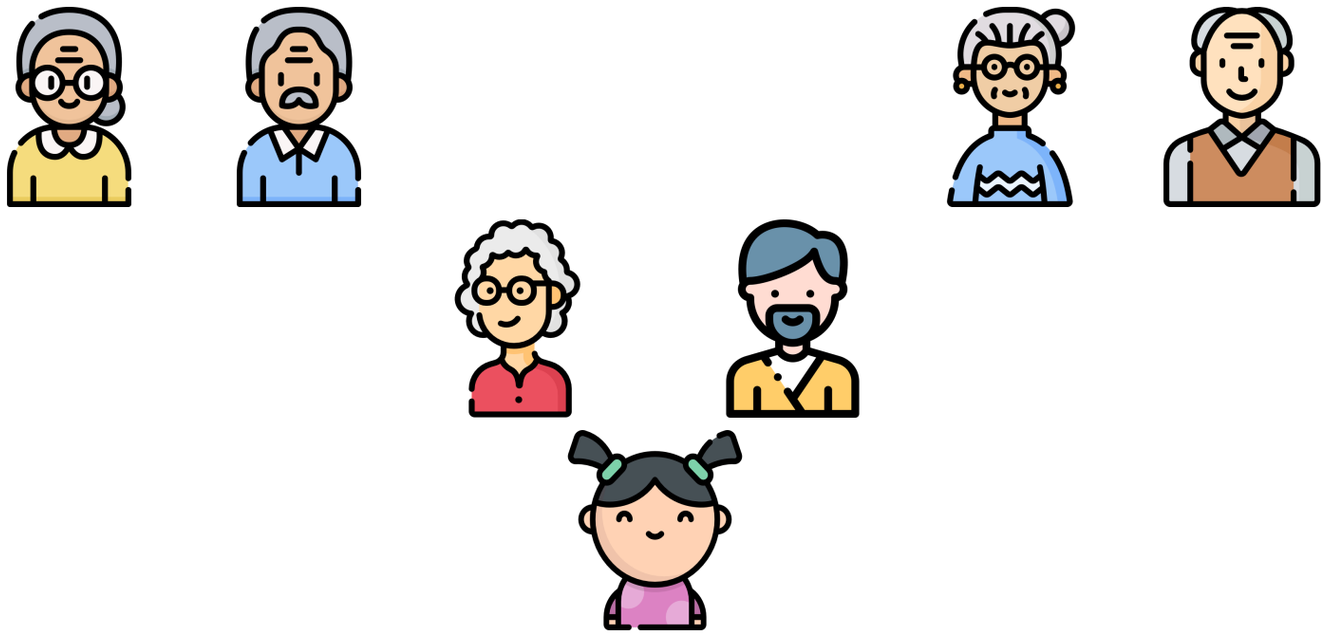
- P was born in Simpleland
- either of P's parents are citizens of Simpleland



Components of Recursion

A person (P) is a citizen of Simpleland if it satisfies any of these conditions:

- P was born in Simpleland **Base case**
- either of P's parents are citizens of Simpleland



Simpleland

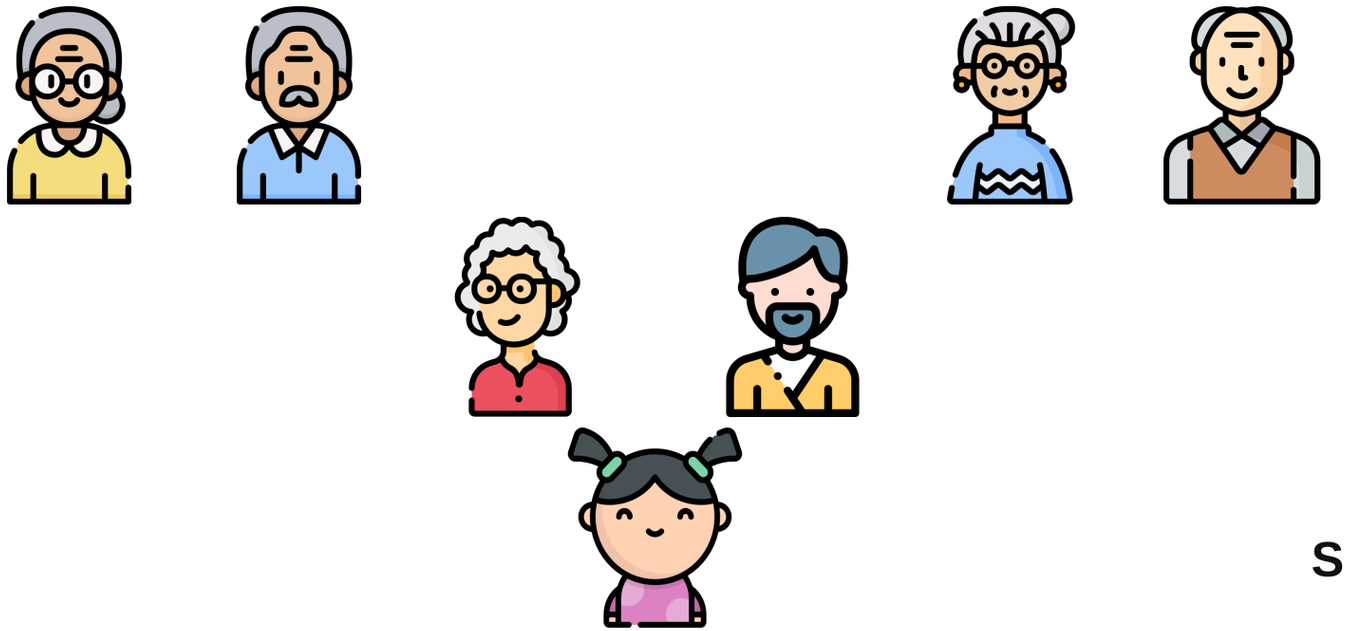


Components of Recursion

A person (P) is a citizen of Simpleland if it satisfies any of these conditions:

- P was born in Simpleland
- either of P's parents are citizens of Simpleland **Recursion case**

Is P a citizen of Simpleland?



Simpleland



Recursion

Universität Konstanz

Definition of recursion

Recursion is a technique in mathematics, logic and computer science to specify a function through itself (a *recursive definition*). The fundamental principle of recursion is to define the value of a function f through previously calculated values of the same function $f(n)$, $f(n-1)$, ...

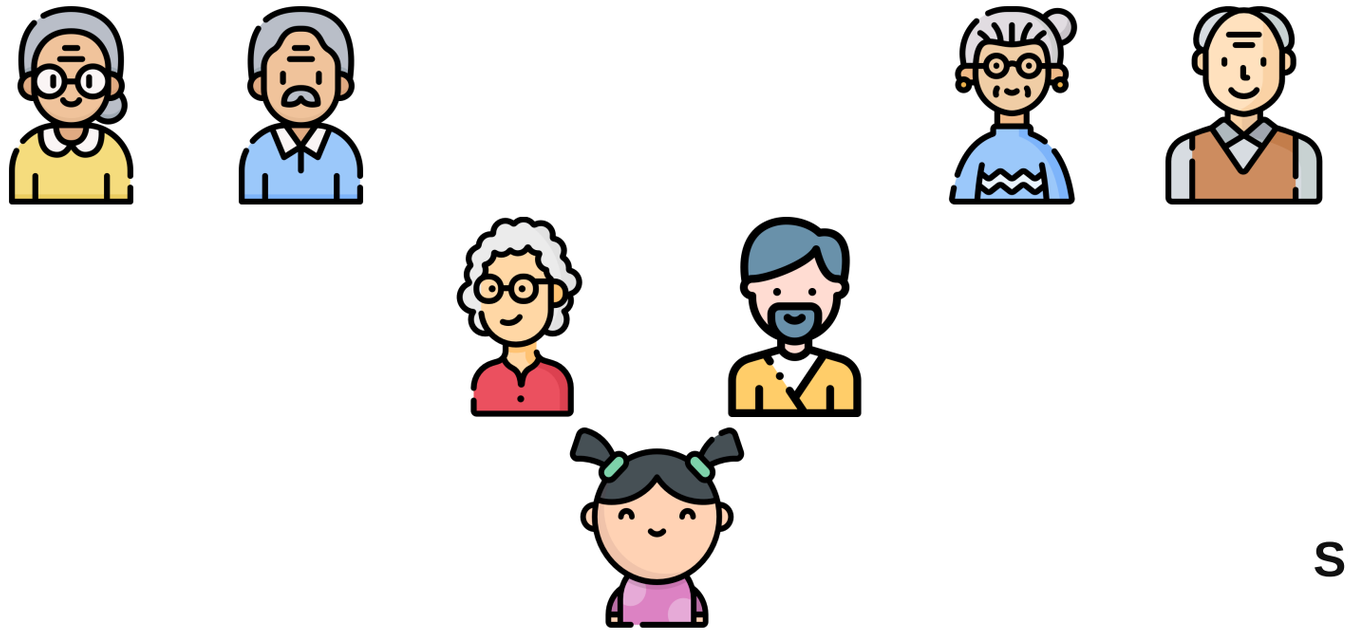
- as long as the start value of the function f is known for sufficiently many arguments (**base cases**), any value of f can be calculated
- a recursive call of the function f keeps calling itself until
  - the function converges to a specific target value
  - or a pre-defined **abort condition** has been met

Issues with Recursion

A person (P) is a citizen of Simpleland if it satisfies any of these conditions:

- P was born in Simpleland
- either of P's parents are citizens of Simpleland

What if we don't know all ancestors of P?



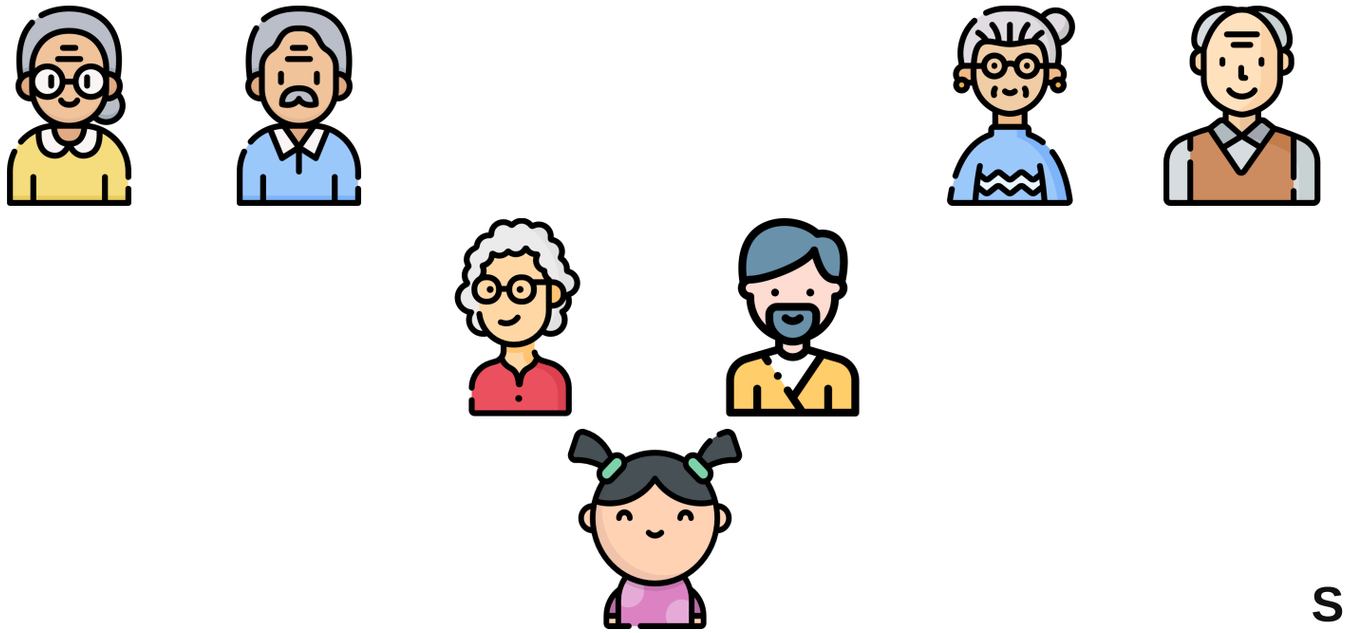
Simpleland



Issues with Recursion

A person (P) is a citizen of Simpleland if it satisfies any of these conditions:

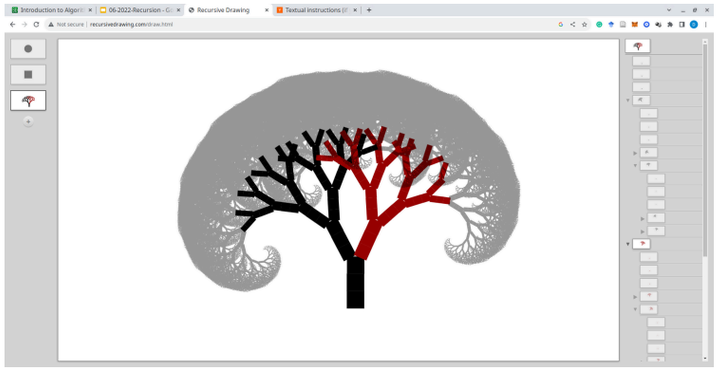
- P was born in Simpleland
- either of P's parents are citizens of Simpleland
- if we don't know P's parents, P is not a citizen of Simpleland **Abort case**

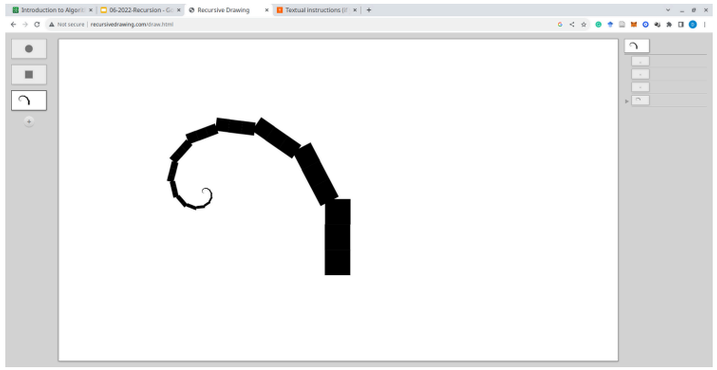


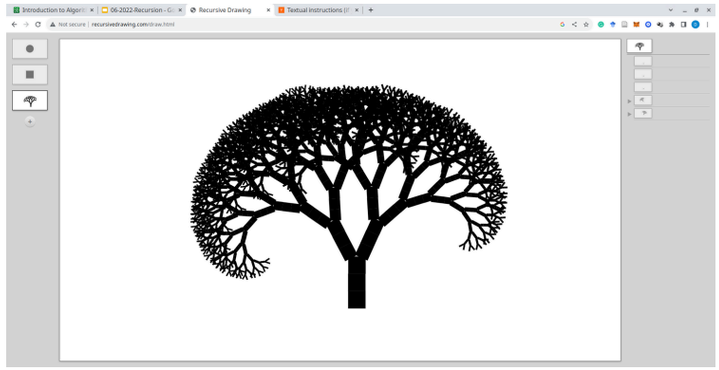
Simpleland



Recursion and Fractals







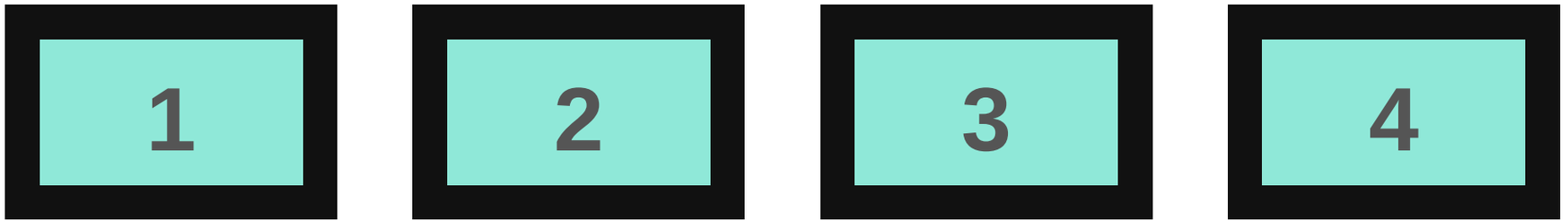
http://recursivedrawing.com/

Factorial: An Example

How many different ways exist for arranging the four diners?

Slots

Choices



Factorial: An Example

How many different ways exist for arranging the four diners?

Slots

Choices



4 * 3 * 2 * 1

Factorial: An Example

How many different ways exist for arranging the four diners?

Slots

Choices



4 * 3 * 2 * 1

$$4! = 4 \cdot 3 \cdot 2 \cdot 1 = 24 \text{ arrangements}$$

Factorial: A Recursive Definition

Iterative definition: $n! = n\,(n-1)\,(n-2)\ldots(1)$

Recursive definition: $n! = n\,(n-1)!$

Factorial: A Recursive Definition

Iterative definition: $n! = n\,(n-1)\,(n-2)\ldots(1)$

Recursive definition: $n! = n\,(n-1)!$

$$n! = \begin{cases} 1 & \text{if } n = 1 \quad (\text{base case}) \\ n \cdot (n-1)! & \text{if } n > 1 \quad (\text{recursion case}) \end{cases}$$

Factorial: Iterative Implementation

Procedure ITERATIVE-FACTORIAL(n)

Inputs:

n: a positive integer

Result: the factorial of n

$result \leftarrow 1$

While $n > 1$:

    $result \leftarrow result \cdot n$

    $n \leftarrow n - 1$

return *result*

$$n! = n\,(n-1)\,(n-2)\ldots(1)$$

Factorial: Recursive Implementation

Universität Konstanz

Procedure RECURSIVE-FACTORIAL(n)

Inputs:

n: a positive integer

Result: the factorial of n

If $n = 1$

then return 1

else return $n \cdot$ RECURSIVE-FACTORIAL(n - 1)

$$n! = n\,(n-1)! \qquad 1! = 1$$

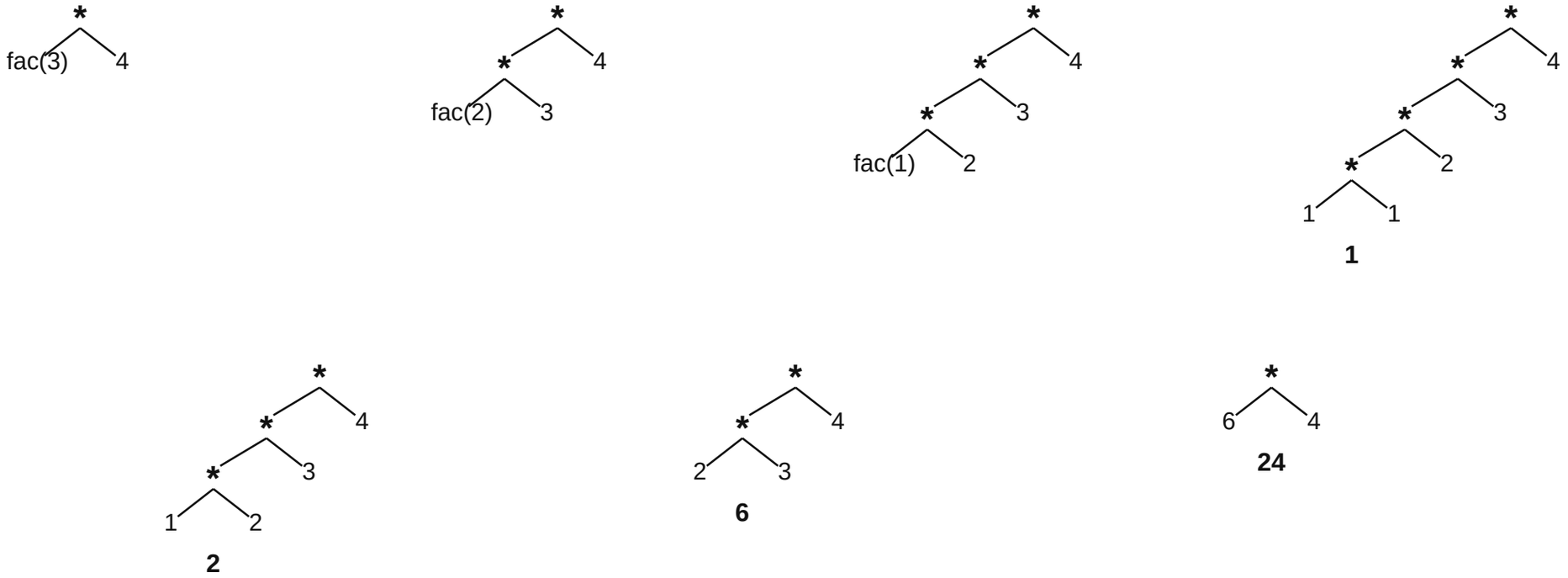
Recursive factorial in Python


```python
# assumes input is a positive integer
# returns n!
def recursive_factorial(n):
    if n == 1:
        return 1
    else:
        return n * recursive_factorial(n-1)
```

$$n! = n\,(n-1)! \quad 1! = 1$$

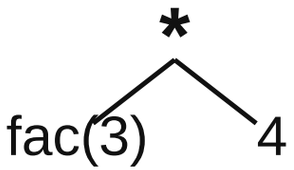
Factorial: Recursive Execution

- Calculation can be represented as a tree
  - example: `fac(4)`



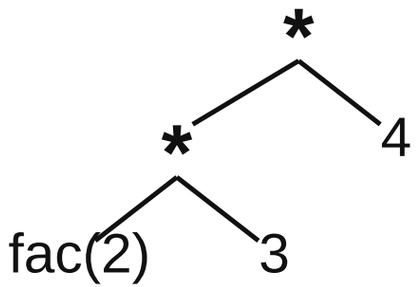
Factorial: Recursive Execution

- Calculation can be represented as a tree
  - example: `fac(4)`



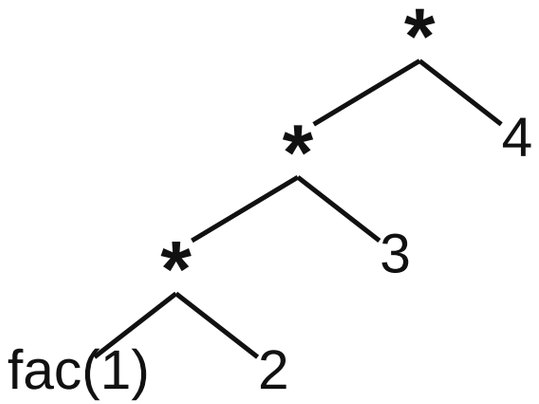
Factorial: Recursive Execution

- Calculation can be represented as a tree
  - example: `fac(4)`



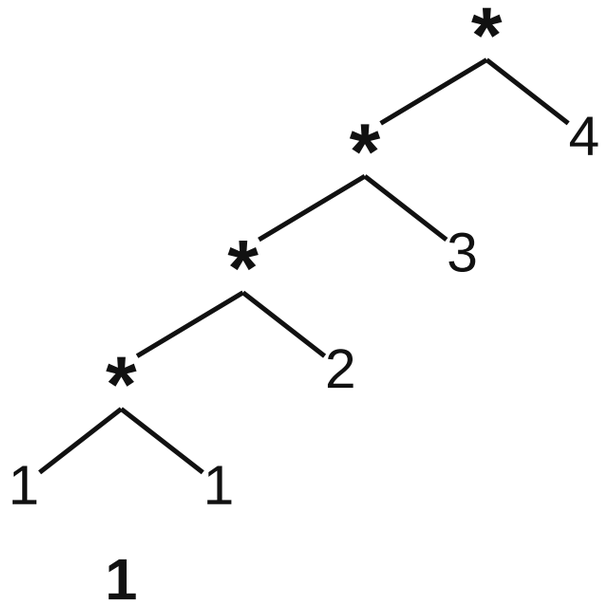
Factorial: Recursive Execution

- Calculation can be represented as a tree
  - example: `fac(4)`



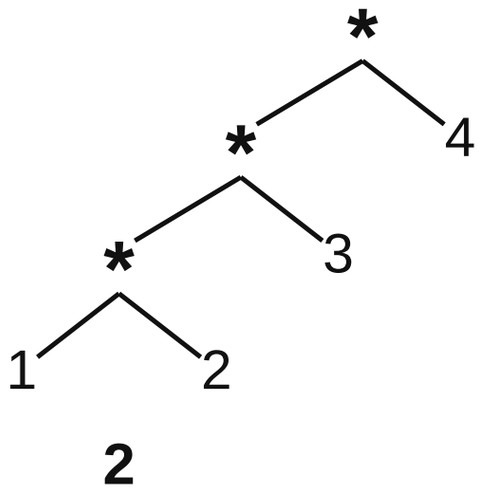
Factorial: Recursive Execution

- Calculation can be represented as a tree
  - example: `fac(4)`



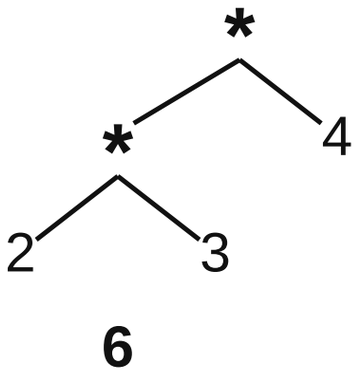
Factorial: Recursive Execution

- Calculation can be represented as a tree
  - example: `fac(4)`



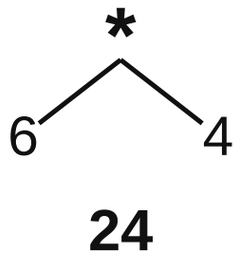
Factorial: Recursive Execution

- Calculation can be represented as a tree
  - example: `fac(4)`



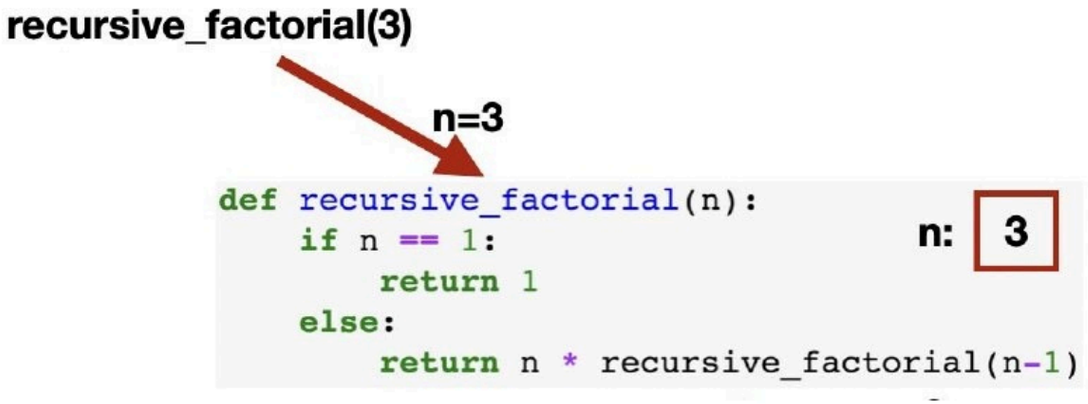
Factorial: Recursive Execution

- Calculation can be represented as a tree
  - example: `fac(4)`

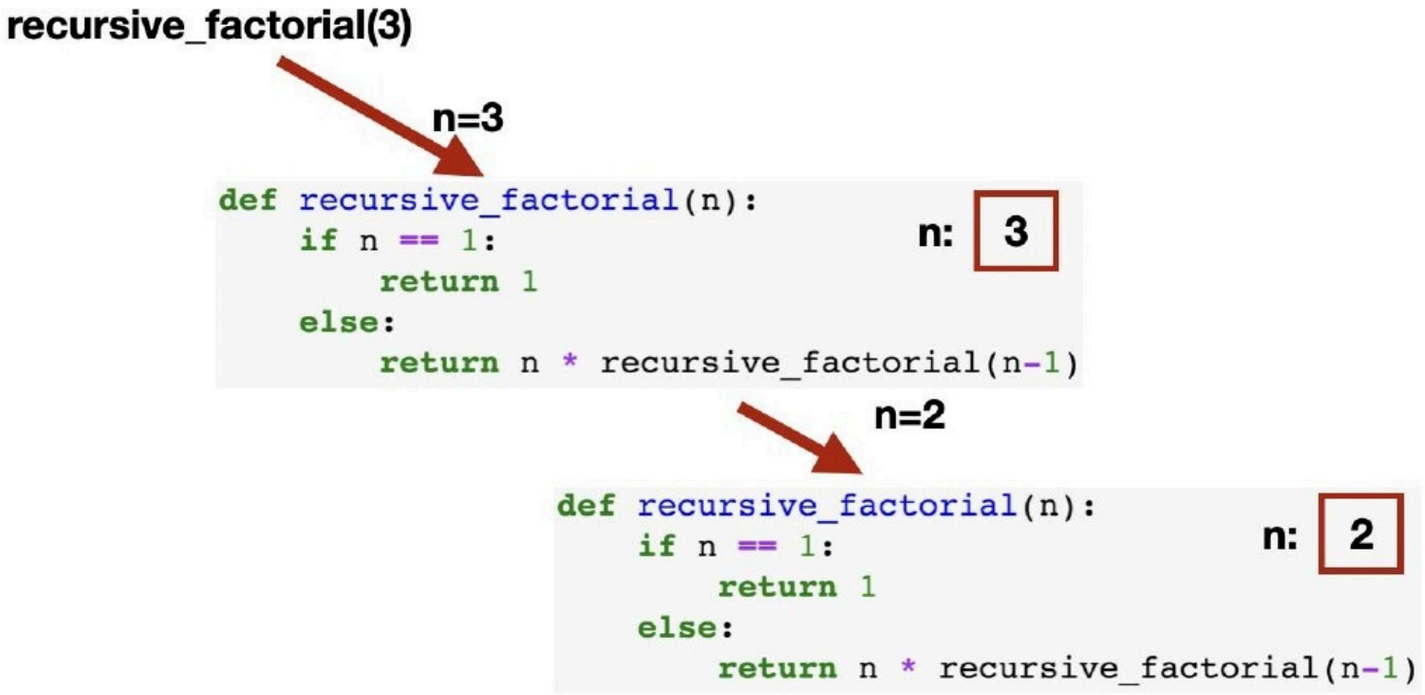


$\Rightarrow$ `fac(4)` = 24

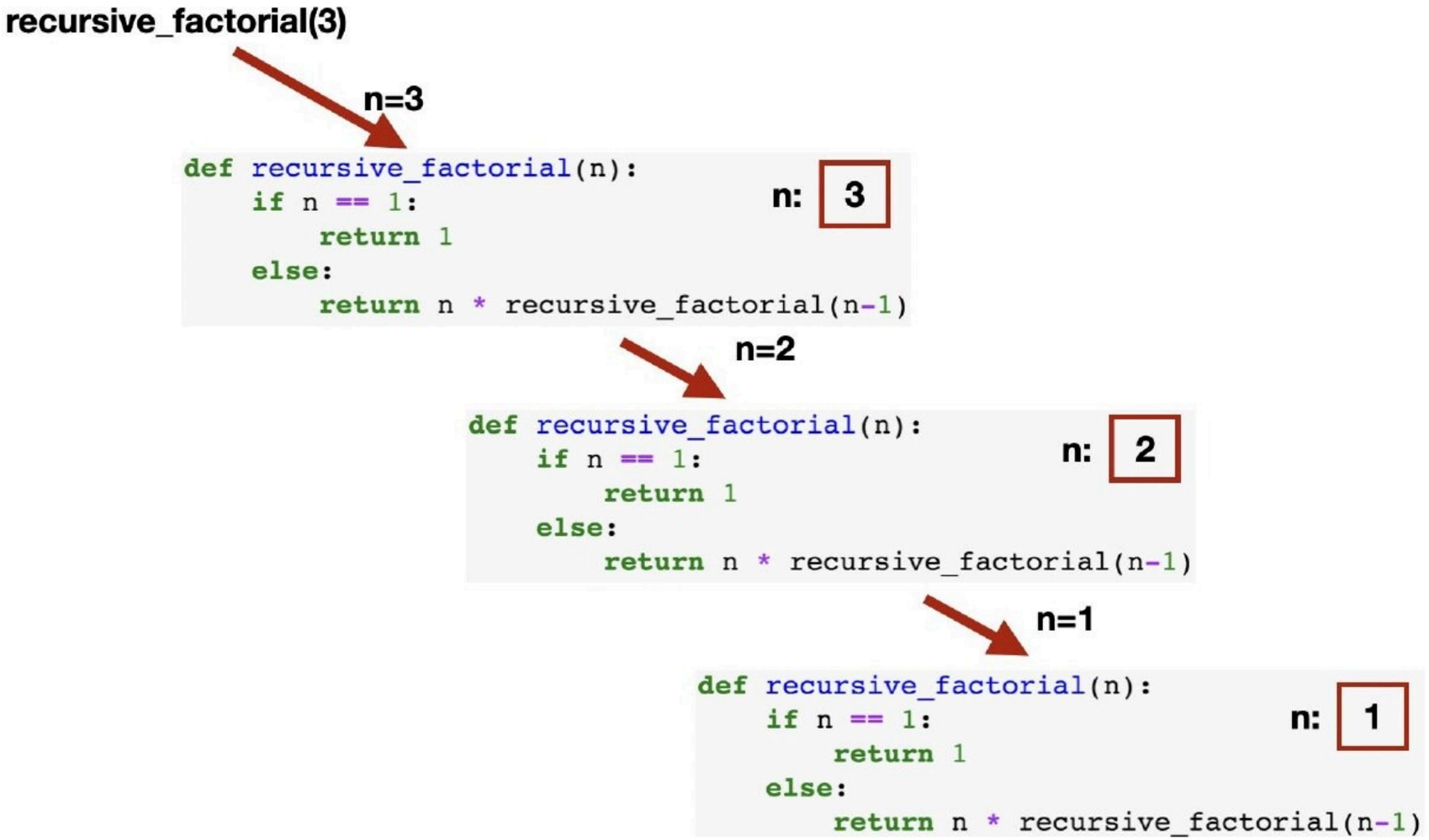
Factorial: Recursive Execution



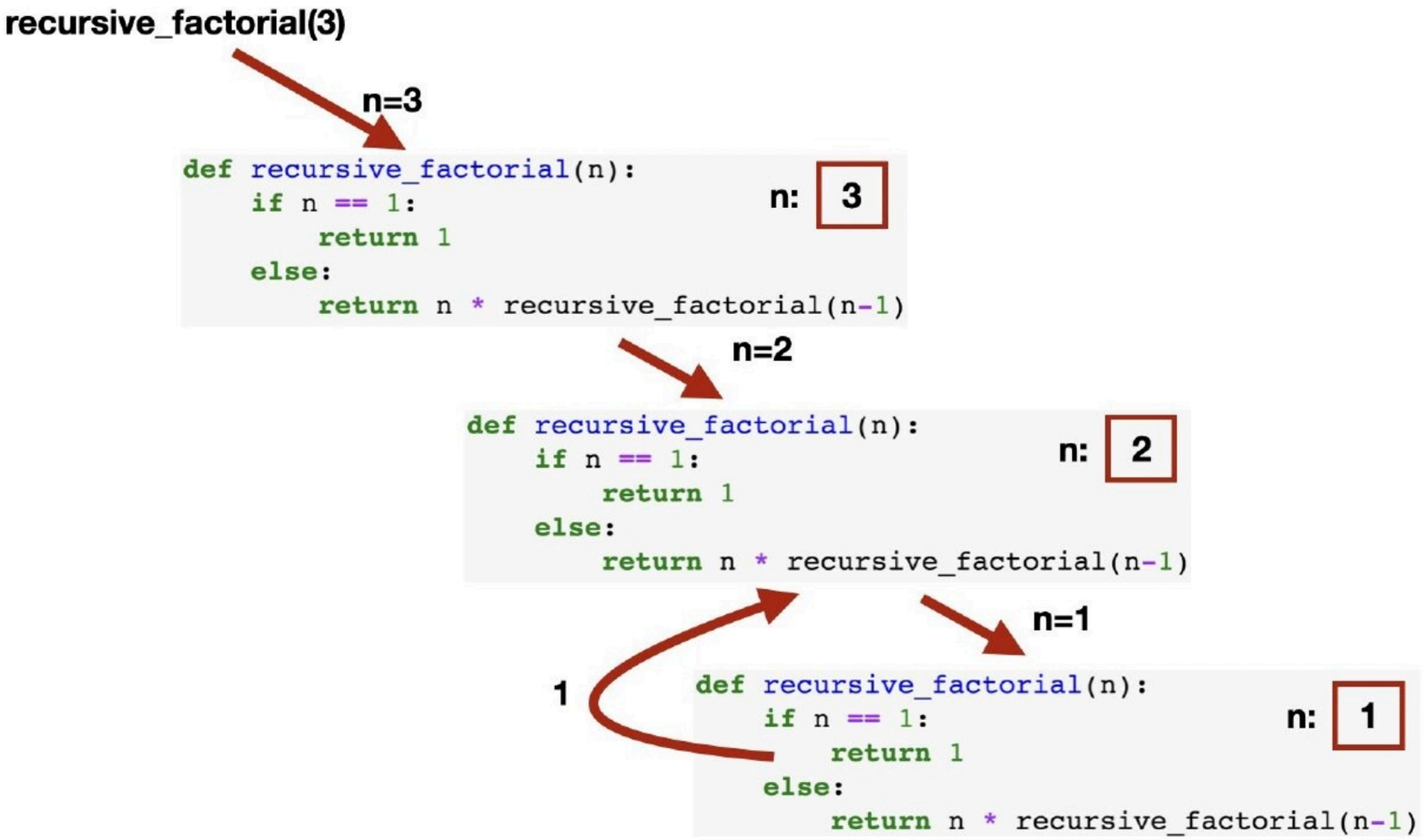
Factorial: Recursive Execution



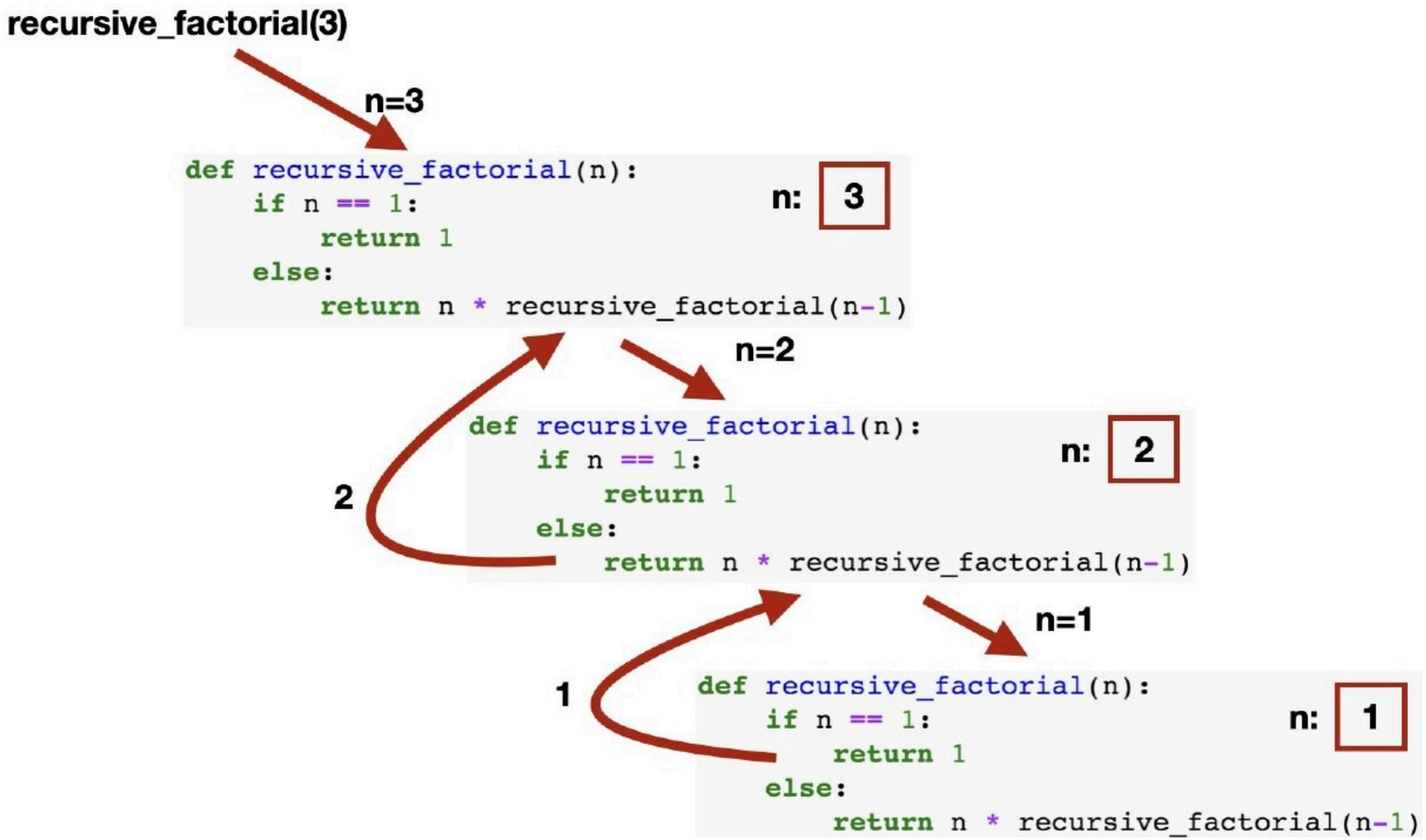
Factorial: Recursive Execution



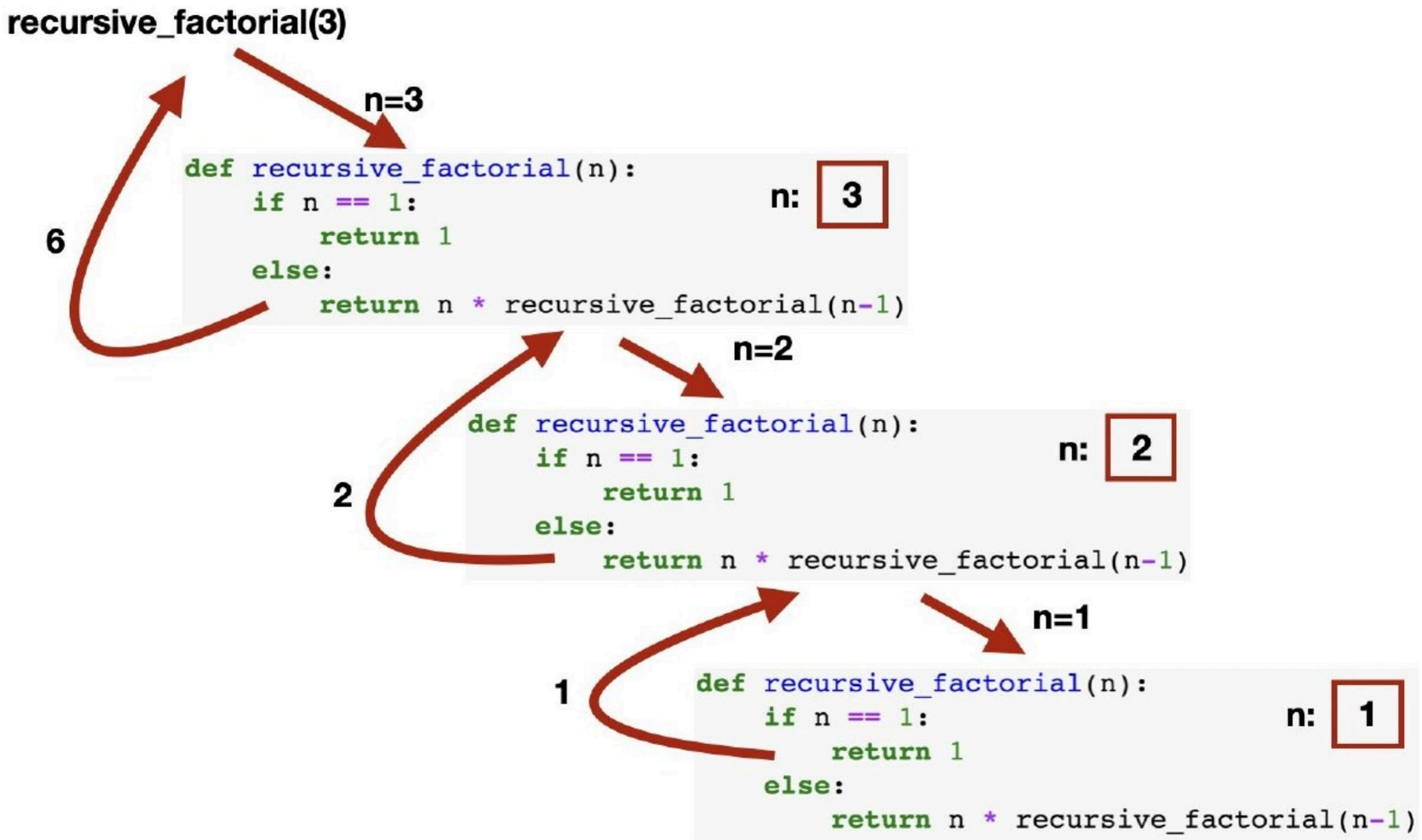
Factorial: Recursive Execution



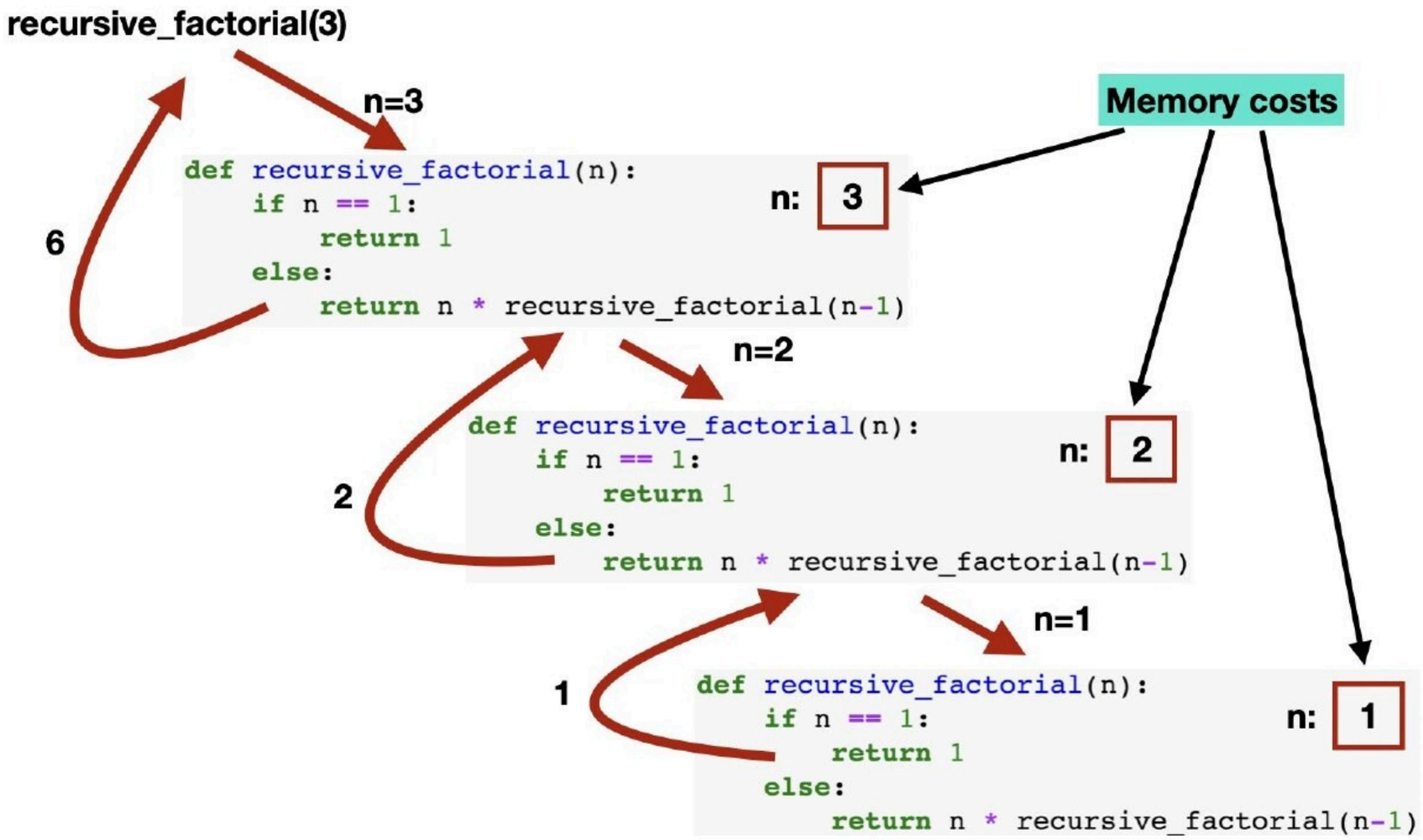
Factorial: Recursive Execution



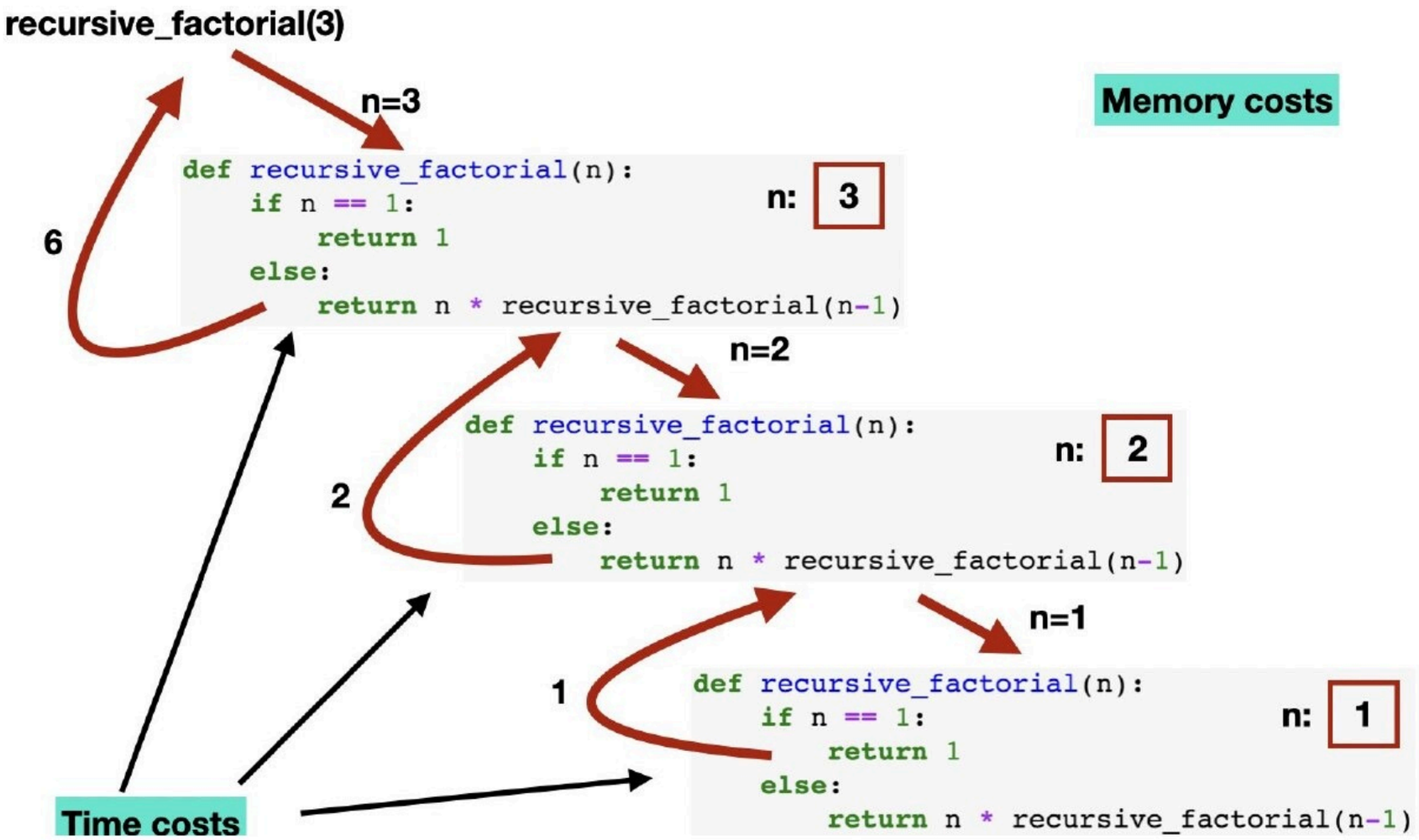
Factorial: Recursive Execution



Factorial: Recursive Execution



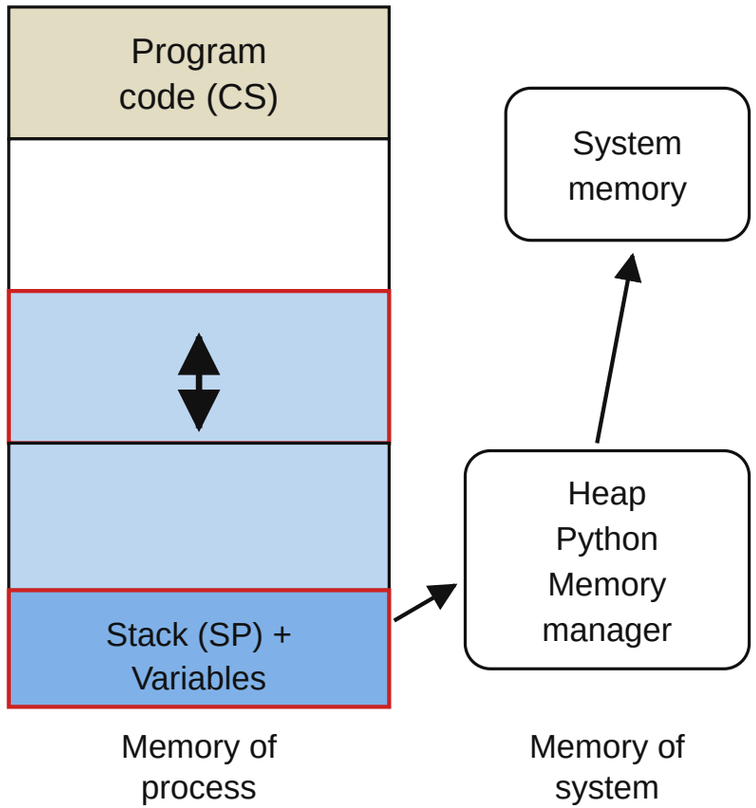
Factorial: Recursive Execution



Intermission: Memory of a running program

A program being run in a machine is called a **process**. Processes require memory to run, which is organized as follows:

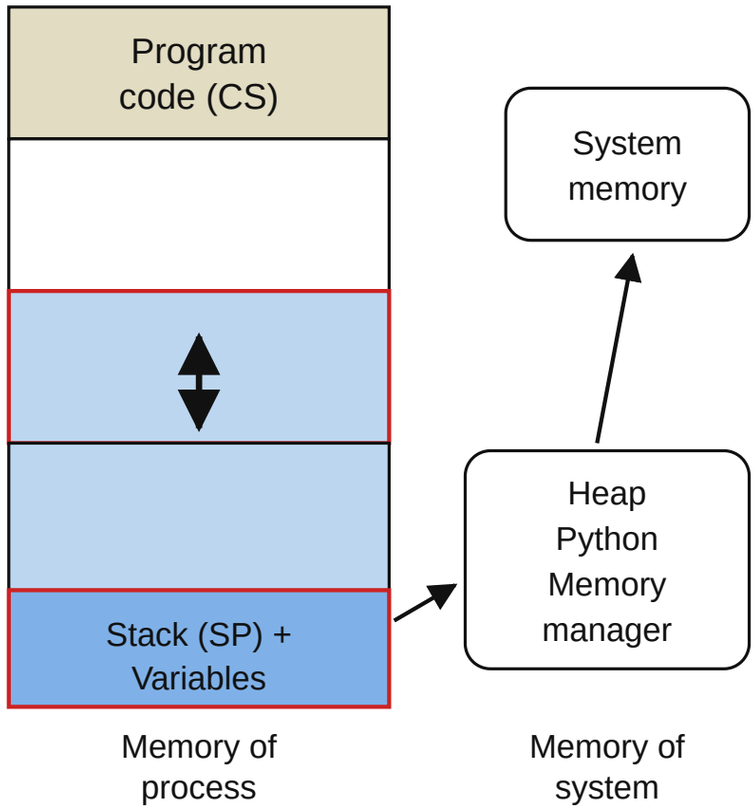
- Code Segment (CS)
- Stack Segment (SS)
- Data Segment or Heap (DS)



Memory of a running program

A program being run in a machine is called a **process**. Processes require memory to run, which is organized as follows:

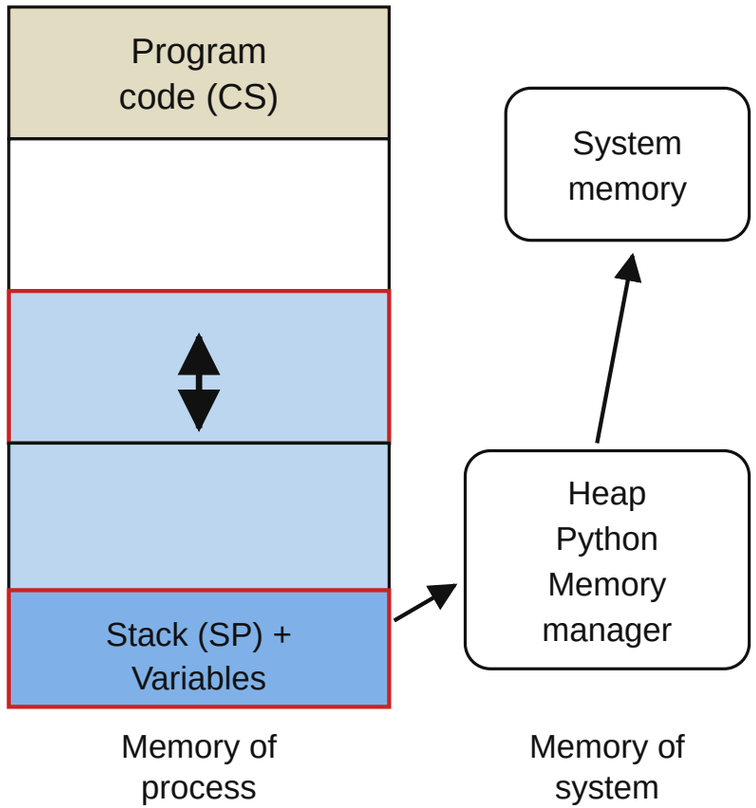
- Code Segment (CS)
  - contains executable instructions, i.e., a program loaded from hard disc
  - typically read-only and of fixed size
- Stack Segment (SS)
- Data Segment or Heap (DS)



Memory of a running program

A program being run in a machine is called a **process**. Processes require memory to run, which is organized as follows:

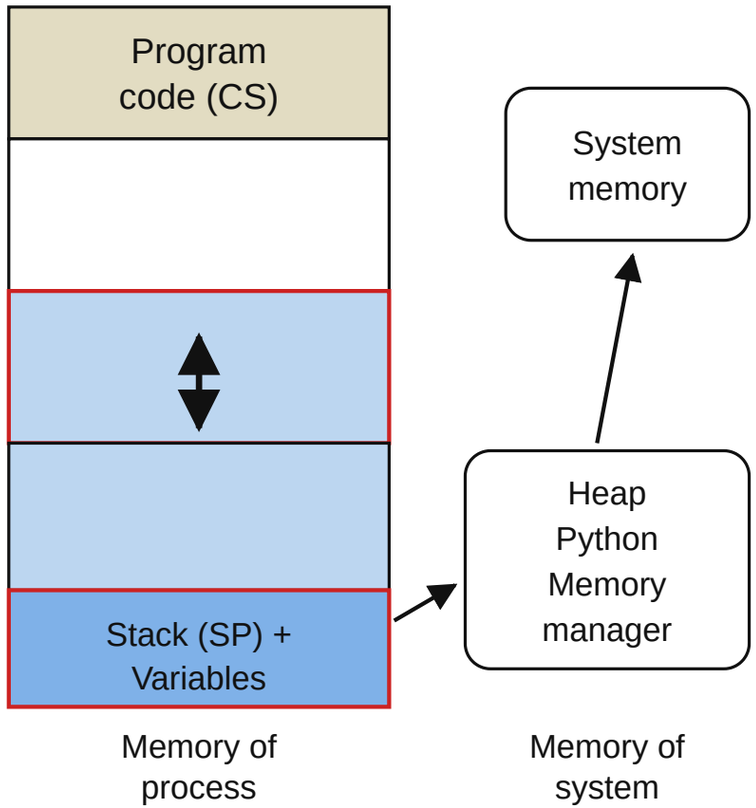
- Code Segment (CS)
- Stack Segment (SS)
  - contains local variables, parameters, return address
  - typically no execute (NX)
  - limited in size, e.g. 8 MB for Linux, 1 MB for Windows and Java
- Data Segment or Heap (DS)



Memory of a running program

A program being run in a machine is called a **process**. Processes require memory to run, which is organized as follows:

- Code Segment (CS)
- Stack Segment (SS)
- Data Segment or Heap (DS)
  - contains global data; Python manages all objects in a (private) heap
  - typically no execute (NX)
  - size is limited by main memory (RAM) or by explicit parameter when run



Recursion: Implementation in memory

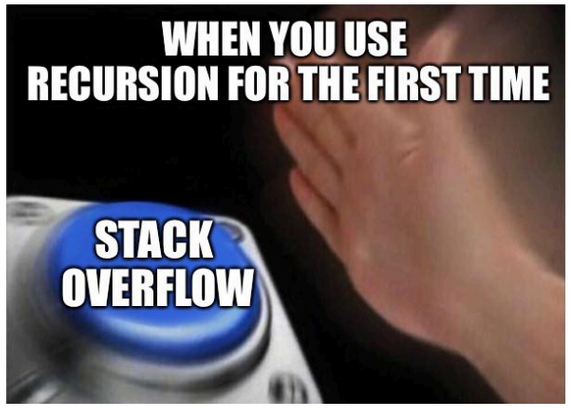
- Recursion uses the **program stack**:
  - parameters
  - return address
  - local variables
  - return values
- stack grows and shrinks automatically with every method call
  - BUT has a maximum size: **stack overflows** are possible!
  - Warning: recursion in functions easily leads to stack overflow

Memes warned you about recursion

3 Steps to being a good Programmer

1. Avoid recursion
2. Repeat steps 1 and 2
3. Always have an exit condition
4. Beware of being off by one

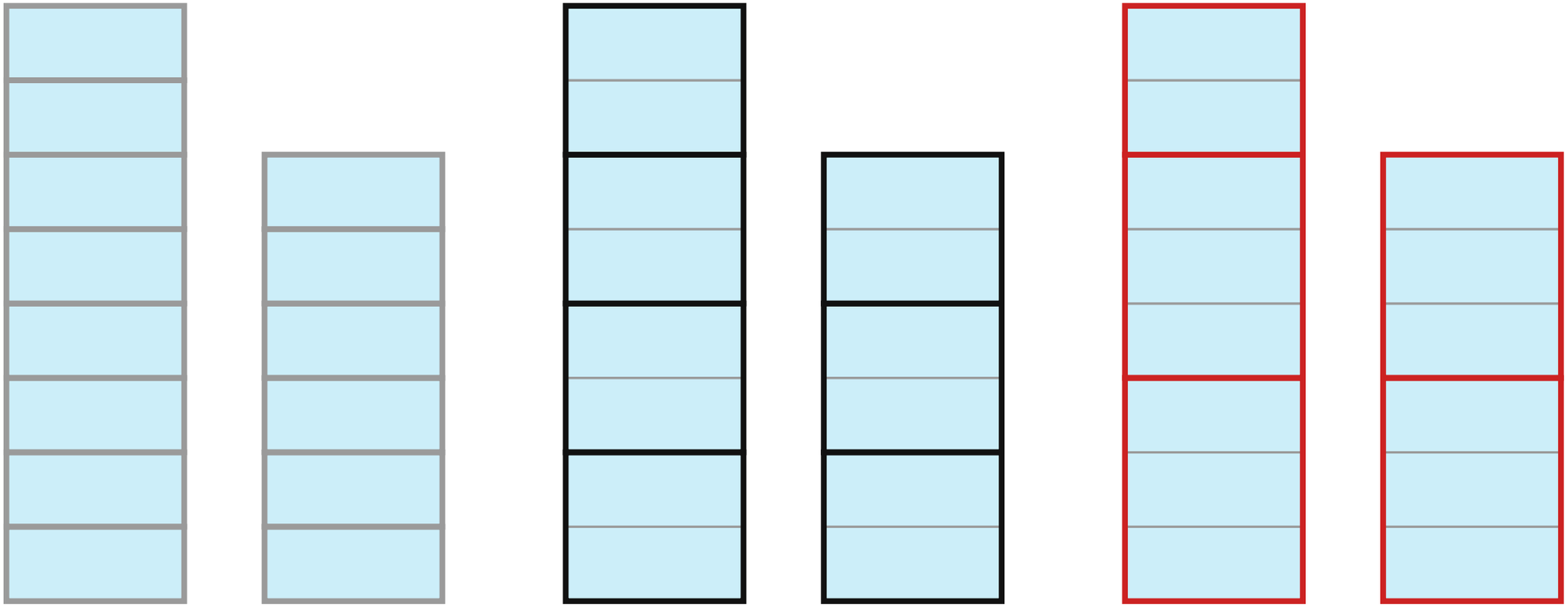




Outline

1. Recursion
2. **Recursive Algorithms**
3. Dynamic Programming

The Greatest Common Divisor



Euclid's algorithm

Uses the mathematical property: $\gcd(a, b) = \gcd(b,\ a \bmod b)$

Procedure GCD(a, b)

Inputs:

a, b: natural numbers (a, $b \in \mathbb{N}$) with $a \geq b$

Result: the greatest common divisor of a and b

If $b = 0$

then return a

else return GCD(b, a **mod** b)

Euclid's algorithm

Uses the mathematical property: $\gcd(a, b) = \gcd(b,\ a \bmod b)$

Procedure GCD(a, b)

Inputs:

a, b: natural numbers (a, $b \in \mathbb{N}$) with $a \geq b$

Result: the greatest common divisor of a and b

If $b = 0$

then return a

else return GCD(b, a **mod** b)

- $\gcd(270, 192)$
  - $270 \bmod 192 = 78$
  - $\gcd(192, 78)$
    - $192 \bmod 78 = 36$
    - $\gcd(78, 36)$
      - $78 \bmod 36 = 6$
      - $\gcd(36, 6)$
        - $36 \bmod 6 = 0$
        - $\gcd(6, 0)$
          - **return 6**

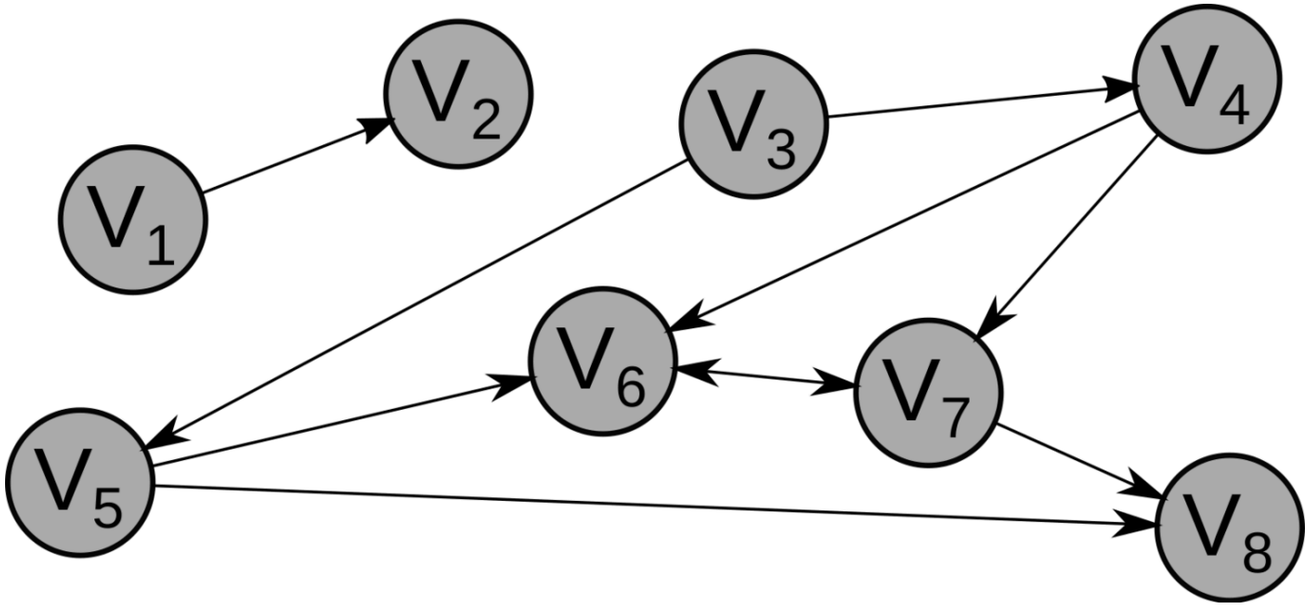
Euclid and the GCD

- Euclid (~300 BC) Greek mathematician
- “Euclid of Alexandria” but little is actually known about his life, neither birth year nor place
- Commonly known as the father of geometry
- Euclid’s most influential book is *Elements*, which includes the algorithm to calculate the gcd
- The original version of the algorithm did subtractions on the terms instead of modulo operations but it was recursive. However, it required more steps
- One of his lesser known books is titled *Data* (Δεδομένα), on how to study given information geometrically



Imagination of how Euclid could have looked like

Depth-First Search in Directed Graphs

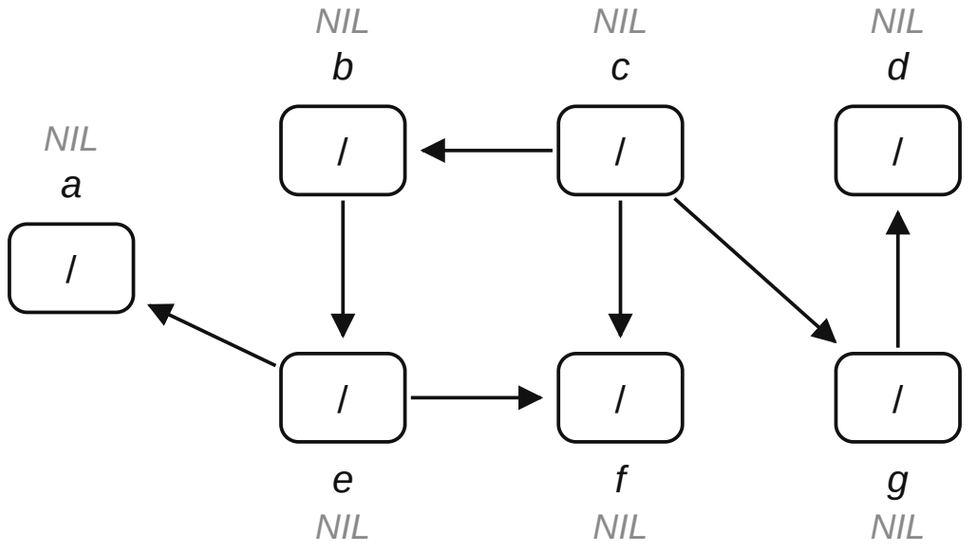


Several problems can be solved with depth-first search, such as:

- Is node x included in the graph?
- Does a graph have a cycle?
- Does the graph have a strongly connected component?

Depth-First Search

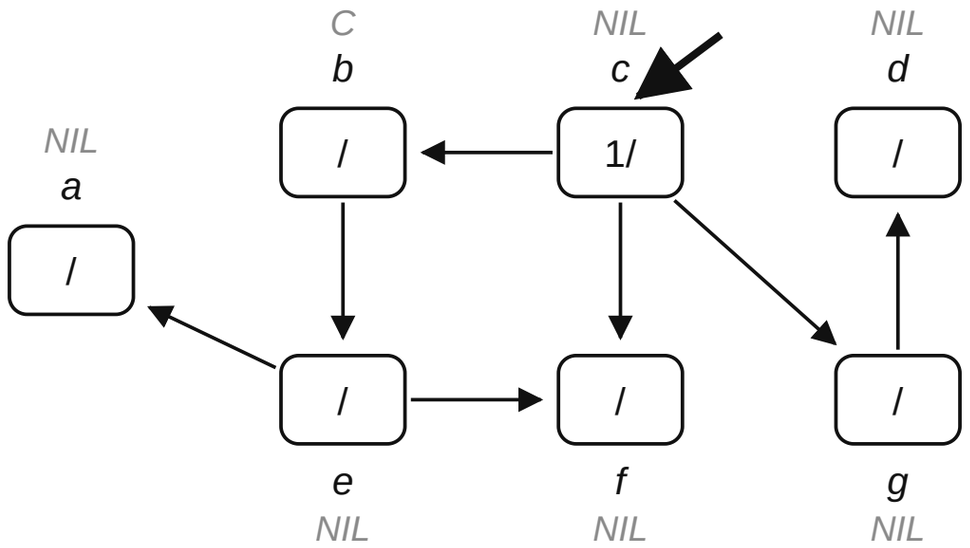
Procedure DFS(*G*)
Inputs:
G: a graph
Result: every vertex stores its discovery time *d*, its finishing time *f* and its parent π
For each vertex *u* in *G.V*:
    color[*u*] ← WHITE
    π[*u*] ← NIL # parent
time ← 0 # global counter
For each vertex *u* in *G.V*:
    If color[*u*] = WHITE
        then DFS-VISIT(*G*, *u*)



Example: the For loop takes vertex *c* first; among neighbors, go alphabetically
time = 0
Enter For-Loop:
call `DFS-VISIT(G, c)`

Depth-First Search

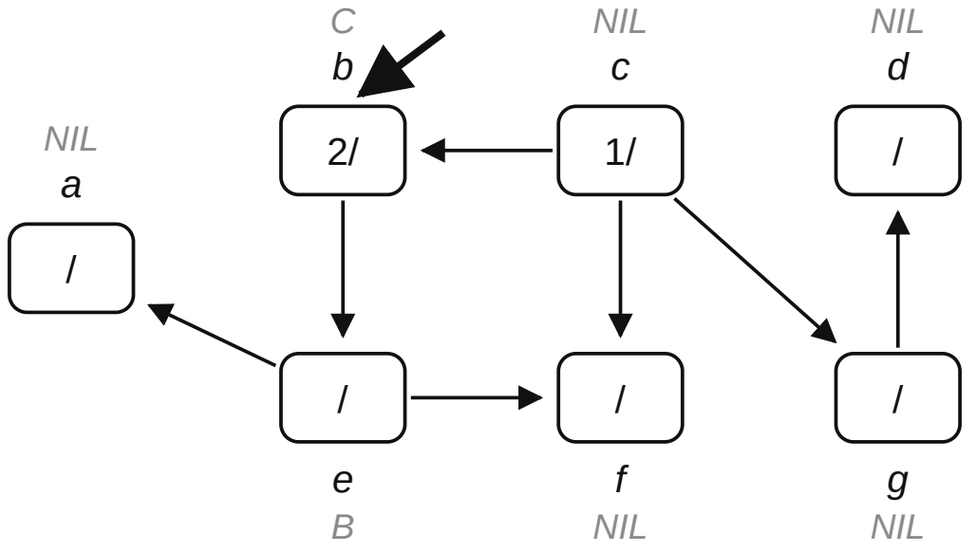
Procedure DFS-VISIT(*G*, *u*)
Inputs:
G: a graph
u: the vertex to be processed
Result: *u* and every vertex reachable from it are visited
time ← *time* + 1
d[*u*] ← *time* # discovered
color[*u*] ← GRAY
For each vertex *v* in neighbors(*u*):
    If color[*v*] = WHITE
        then π[*v*] ← *u*
            DFS-VISIT(*G*, *v*)
color[*u*] ← BLACK
time ← *time* + 1
f[*u*] ← *time* # finished



call `DFS-VISIT(G, c)`
time = 1
Enter For-Loop:
next adjacent white node: b
add parent of b
call `DFS-VISIT(G, b)`

Depth-First Search

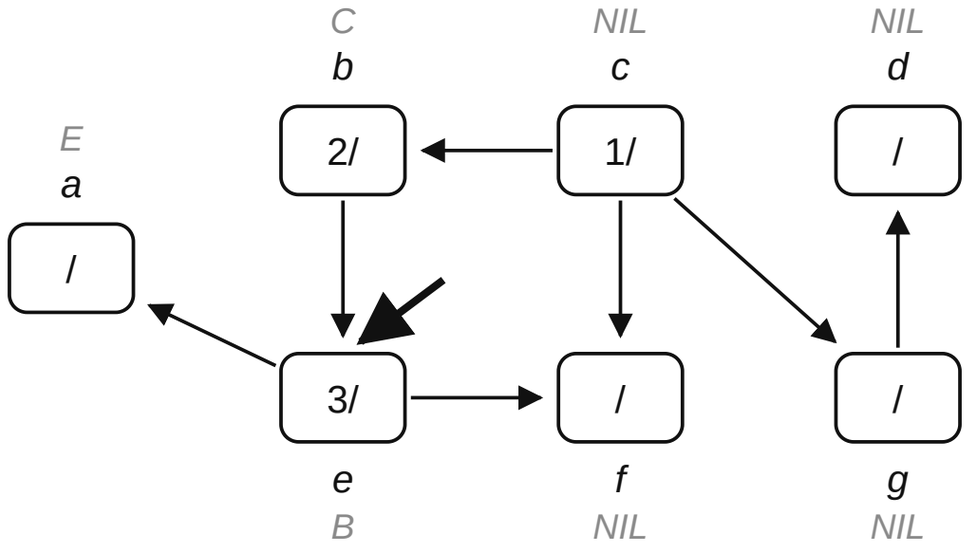
Procedure DFS-VISIT(*G*, *u*)
Inputs:
G: a graph
u: the vertex to be processed
Result: *u* and every vertex reachable from it are visited
time ← *time* + 1
d[*u*] ← *time* # discovered
color[*u*] ← GRAY
For each vertex *v* in neighbors(*u*):
If color[*v*] = WHITE
then π[*v*] ← *u*
DFS-VISIT(*G*, *v*)
color[*u*] ← BLACK
time ← *time* + 1
f[*u*] ← *time* # finished



call `DFS-VISIT(G, b)`
time = 2
Enter For-Loop:
next adjacent white node: e
add parent of e
call `DFS-VISIT(G, e)`

Depth-First Search

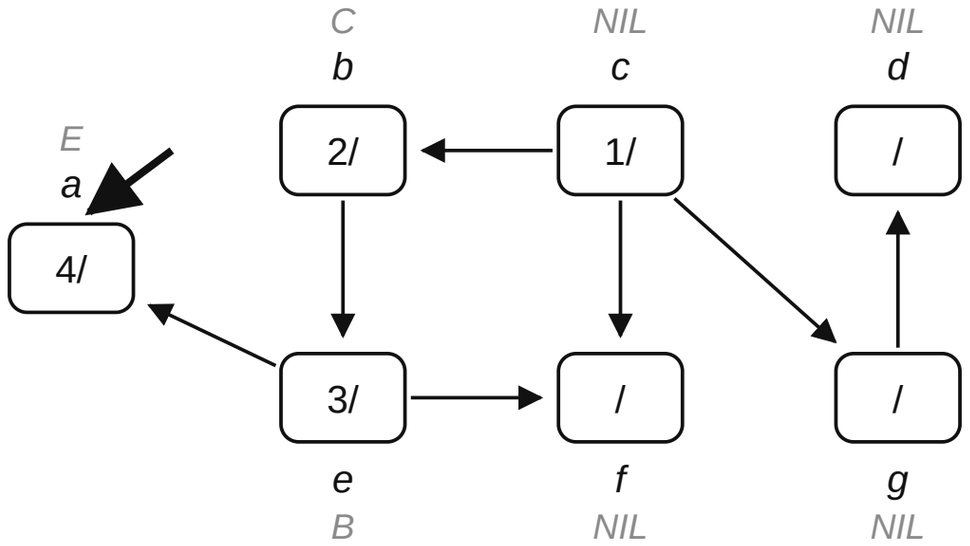
Procedure DFS-VISIT(*G*, *u*)
Inputs:
G: a graph
u: the vertex to be processed
Result: *u* and every vertex reachable from it are visited
time ← *time* + 1
d[*u*] ← *time* # discovered
color[*u*] ← GRAY
For each vertex *v* in neighbors(*u*):
    If color[*v*] = WHITE
        then π[*v*] ← *u*
            DFS-VISIT(*G*, *v*)
color[*u*] ← BLACK
time ← *time* + 1
f[*u*] ← *time* # finished



call `DFS-VISIT(G, e)`
time = 3
Enter For-Loop:
next adjacent white node: a
add parent of a
call `DFS-VISIT(G, a)`

Depth-First Search

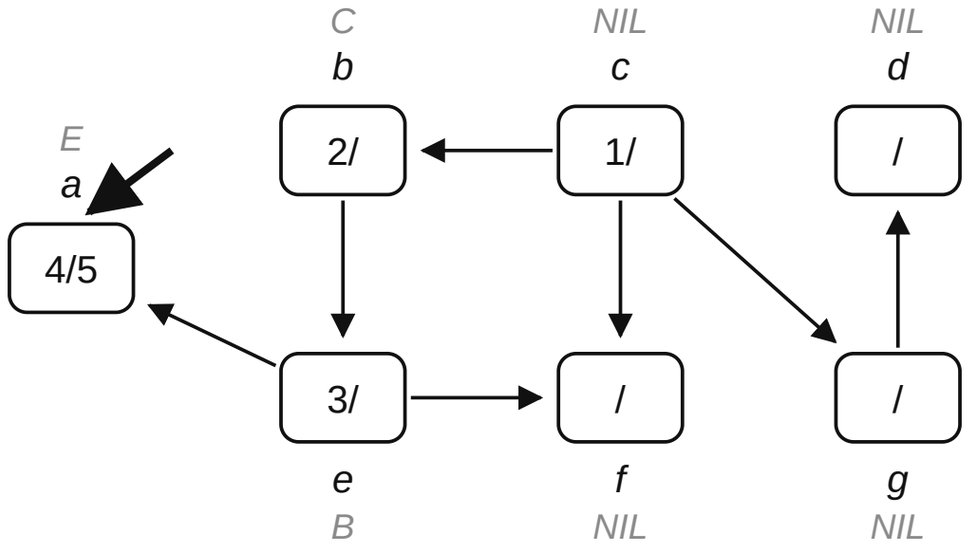
Procedure DFS-VISIT(*G*, *u*)
Inputs:
G: a graph
u: the vertex to be processed
Result: *u* and every vertex reachable from it are visited
time ← *time* + 1
d[*u*] ← *time* # discovered
color[*u*] ← GRAY
For each vertex *v* in neighbors(*u*):
    If color[*v*] = WHITE
        then π[*v*] ← *u*
            DFS-VISIT(*G*, *v*)
color[*u*] ← BLACK
time ← *time* + 1
f[*u*] ← *time* # finished



call `DFS-VISIT(G, a)`
time = 4
Enter For-Loop:
no adjacent white nodes
finish `DFS-VISIT(G, a)`

Depth-First Search

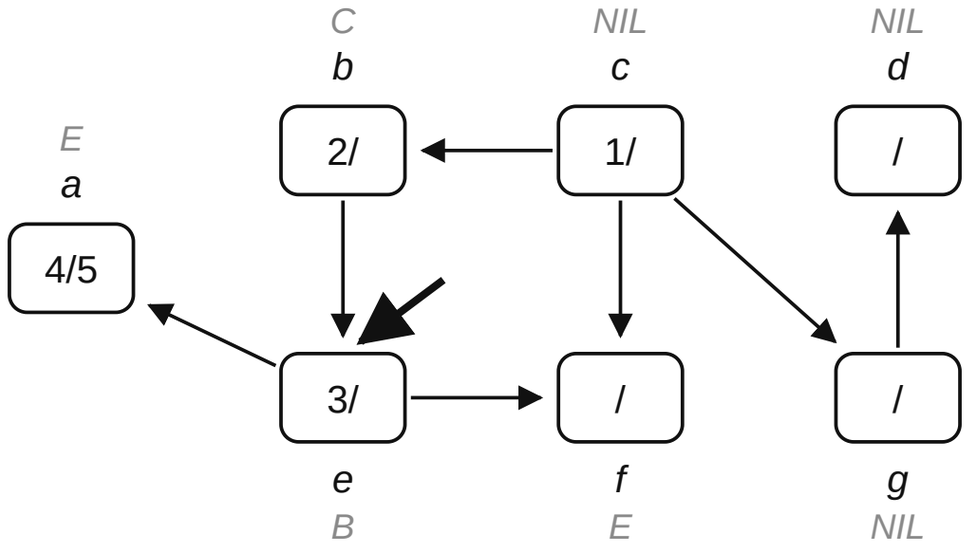
Procedure DFS-VISIT(*G*, *u*)
Inputs:
G: a graph
u: the vertex to be processed
Result: *u* and every vertex reachable from it are visited
time ← *time* + 1
d[*u*] ← *time* # discovered
color[*u*] ← GRAY
For each vertex *v* in neighbors(*u*):
    If color[*v*] = WHITE
        then π[*v*] ← *u*
            DFS-VISIT(*G*, *v*)
color[*u*] ← BLACK
time ← *time* + 1
f[*u*] ← *time* # finished



finish `DFS-VISIT(G, a)`
time = 5
return to For-Loop of `DFS-VISIT(G, e)`

Depth-First Search

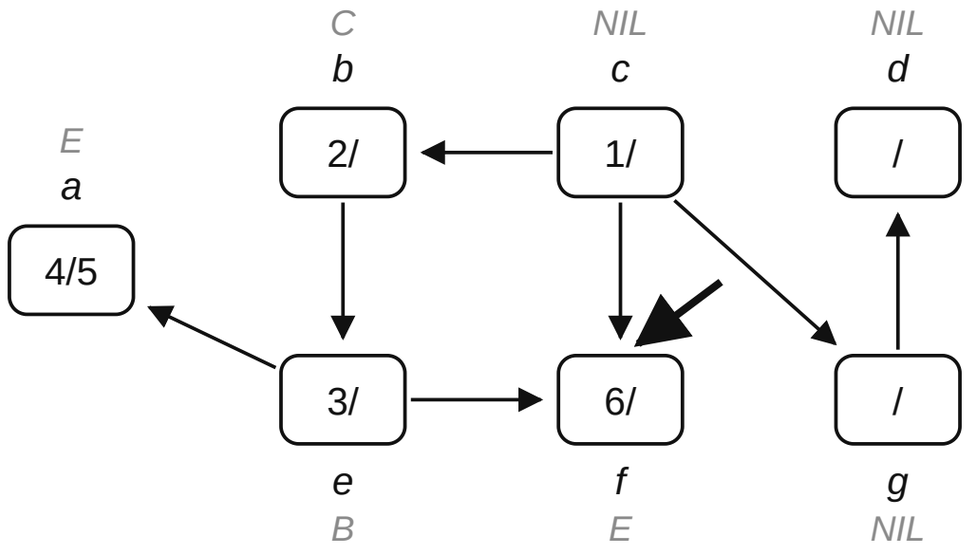
Procedure DFS-VISIT(*G*, *u*)
Inputs:
G: a graph
u: the vertex to be processed
Result: *u* and every vertex reachable from it are visited
time ← *time* + 1
d[*u*] ← *time* # discovered
color[*u*] ← GRAY
For each vertex *v* in neighbors(*u*):
    If color[*v*] = WHITE
        then π[*v*] ← *u*
            DFS-VISIT(*G*, *v*)
color[*u*] ← BLACK
time ← *time* + 1
f[*u*] ← *time* # finished



return to For-Loop of `DFS-VISIT(G, e)`
time = 5
Back in the For-Loop:
next adjacent white node: f
add parent of f
call `DFS-VISIT(G, f)`

Depth-First Search

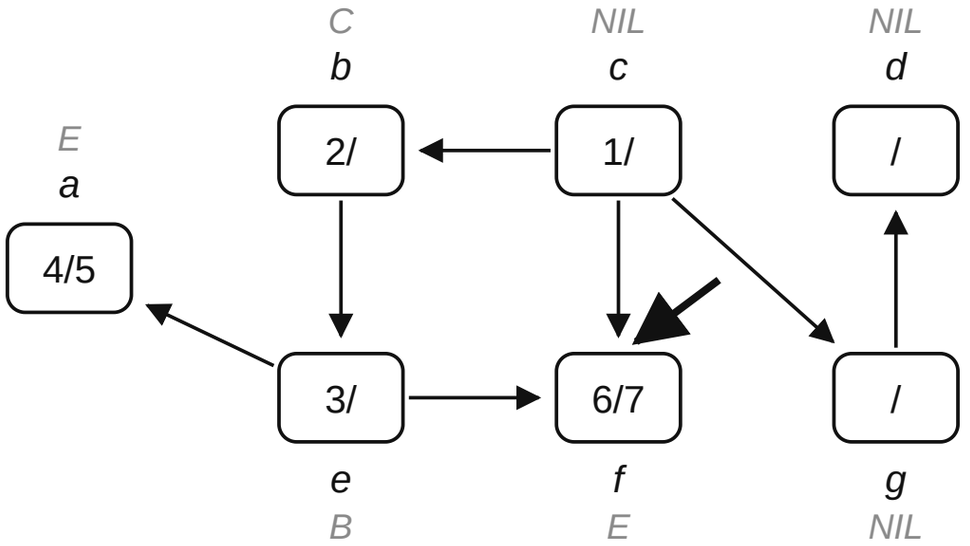
Procedure DFS-VISIT(*G*, *u*)
Inputs:
G: a graph
u: the vertex to be processed
Result: *u* and every vertex reachable from it are visited
time ← *time* + 1
d[*u*] ← *time* # discovered
color[*u*] ← GRAY
For each vertex *v* in neighbors(*u*):
    If color[*v*] = WHITE
        then π[*v*] ← *u*
            DFS-VISIT(*G*, *v*)
color[*u*] ← BLACK
time ← *time* + 1
f[*u*] ← *time* # finished



call `DFS-VISIT(G, f)`
time = 6
Enter the For-Loop:
no adjacent white nodes
finish `DFS-VISIT(G, f)`

Depth-First Search

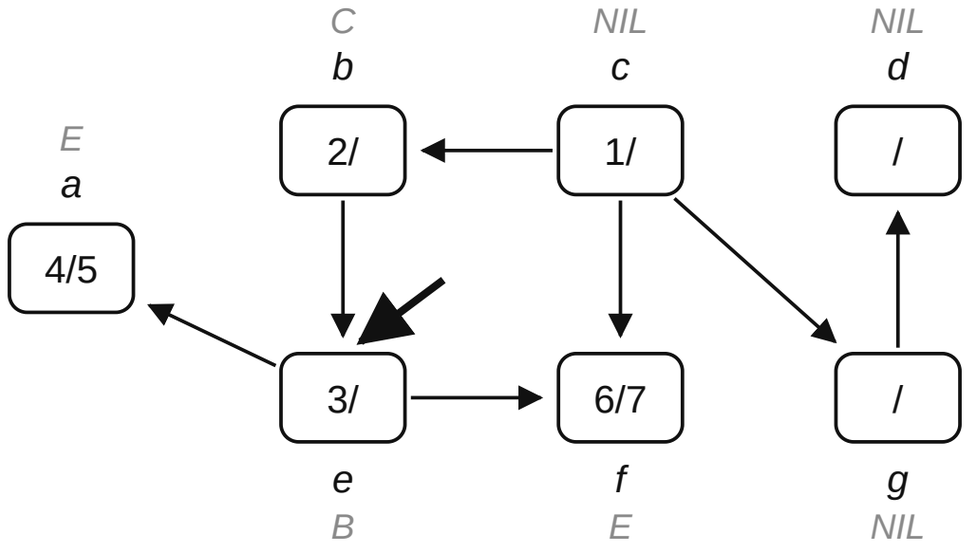
Procedure DFS-VISIT(*G*, *u*)
Inputs:
G: a graph
u: the vertex to be processed
Result: *u* and every vertex reachable from it are visited
time ← *time* + 1
d[*u*] ← *time* # discovered
color[*u*] ← GRAY
For each vertex *v* in neighbors(*u*):
    If color[*v*] = WHITE
        then π[*v*] ← *u*
            DFS-VISIT(*G*, *v*)
color[*u*] ← BLACK
time ← *time* + 1
f[*u*] ← *time* # finished



finish `DFS-VISIT(G, f)`
time = 7
return to For-Loop of `DFS-VISIT(G, e)`

Depth-First Search

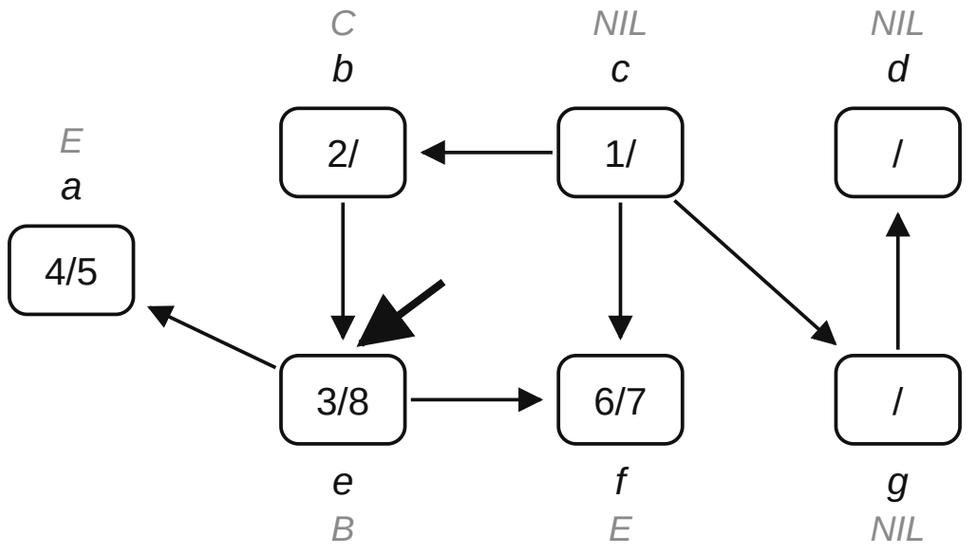
Procedure DFS-VISIT(*G*, *u*)
Inputs:
G: a graph
u: the vertex to be processed
Result: *u* and every vertex reachable from it are visited
time ← *time* + 1
d[*u*] ← *time* # discovered
color[*u*] ← GRAY
For each vertex *v* in neighbors(*u*):
    If color[*v*] = WHITE
        then π[*v*] ← *u*
            DFS-VISIT(*G*, *v*)
color[*u*] ← BLACK
time ← *time* + 1
f[*u*] ← *time* # finished



return to For-Loop of `DFS-VISIT(G, e)`
time = 7
Back in the For-Loop:
no adjacent white nodes
finish `DFS-VISIT(G, e)`

Depth-First Search

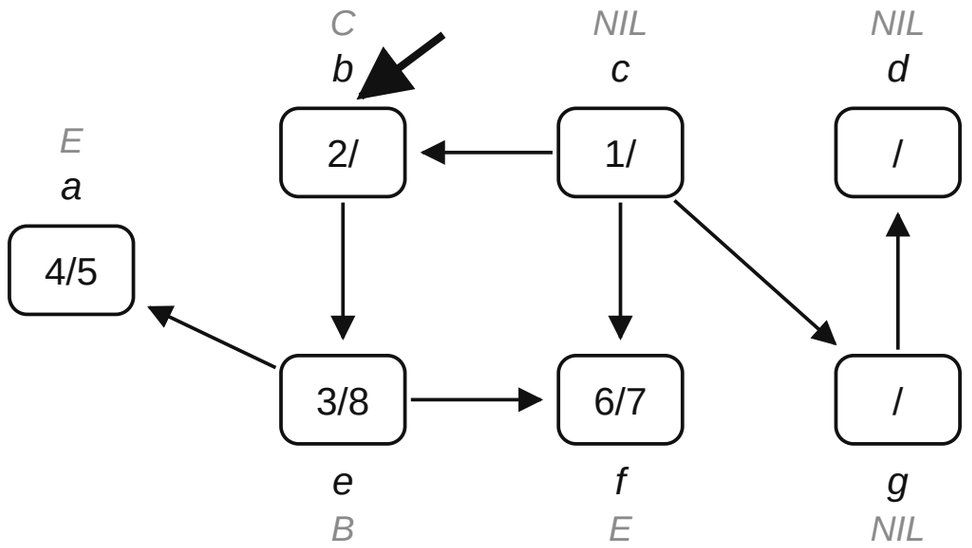
Procedure DFS-VISIT(*G*, *u*)
Inputs:
G: a graph
u: the vertex to be processed
Result: *u* and every vertex reachable from it are visited
time ← *time* + 1
d[*u*] ← *time* # discovered
color[*u*] ← GRAY
For each vertex *v* in neighbors(*u*):
    If color[*v*] = WHITE
        then π[*v*] ← *u*
            DFS-VISIT(*G*, *v*)
color[*u*] ← BLACK
time ← *time* + 1
f[*u*] ← *time* # finished



finish `DFS-VISIT(G, e)`
time = 8
return to For-Loop of `DFS-VISIT(G, b)`

Depth-First Search

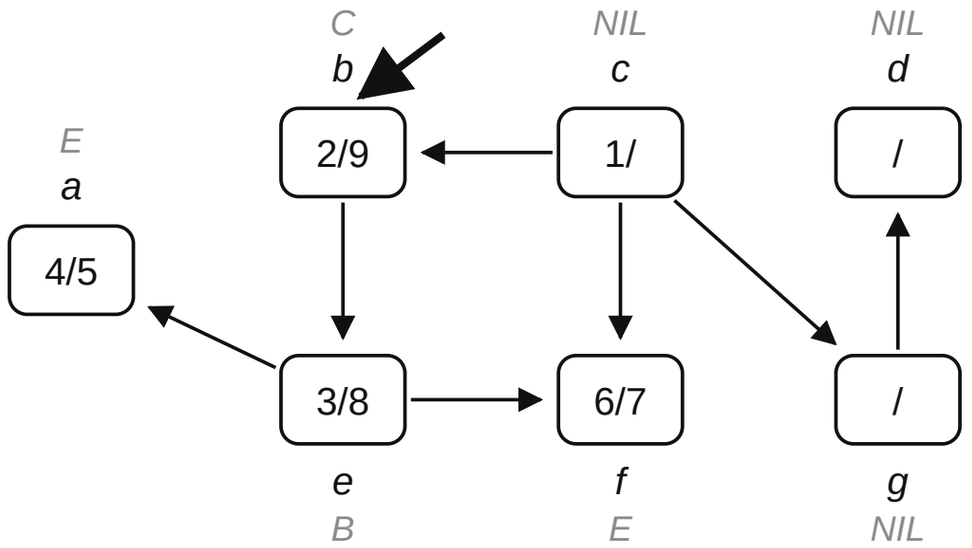
Procedure DFS-VISIT(*G*, *u*)
Inputs:
G: a graph
u: the vertex to be processed
Result: *u* and every vertex reachable from it are visited
time ← *time* + 1
d[*u*] ← *time* # discovered
color[*u*] ← GRAY
For each vertex *v* in neighbors(*u*):
    If color[*v*] = WHITE
        then π[*v*] ← *u*
            DFS-VISIT(*G*, *v*)
color[*u*] ← BLACK
time ← *time* + 1
f[*u*] ← *time* # finished



return to For-Loop of `DFS-VISIT(G, b)`
time = 8
Back in the For-Loop:
no adjacent white nodes
finish `DFS-VISIT(G, b)`

Depth-First Search

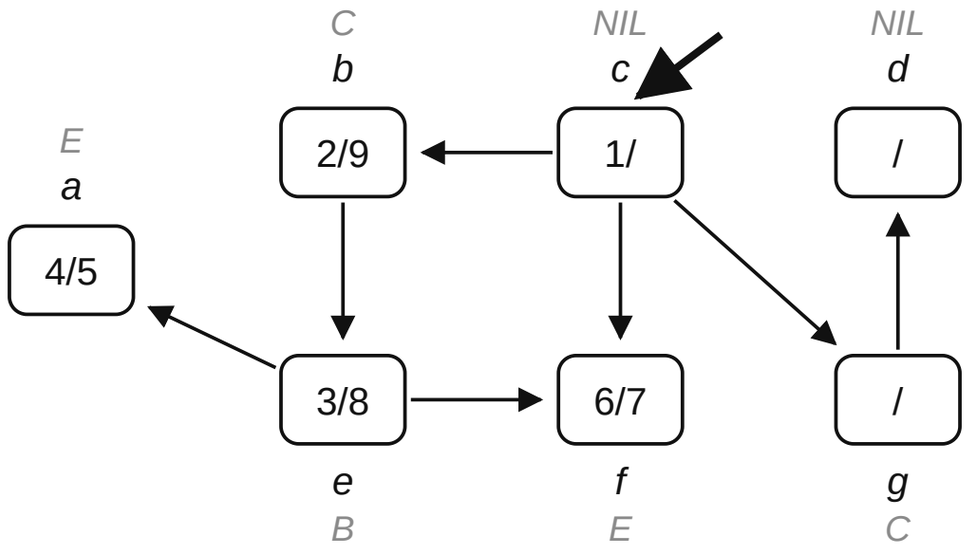
Procedure DFS-VISIT(*G*, *u*)
Inputs:
G: a graph
u: the vertex to be processed
Result: *u* and every vertex reachable from it are visited
time ← *time* + 1
d[*u*] ← *time* # discovered
color[*u*] ← GRAY
For each vertex *v* in neighbors(*u*):
    If color[*v*] = WHITE
        then π[*v*] ← *u*
            DFS-VISIT(*G*, *v*)
color[*u*] ← BLACK
time ← *time* + 1
f[*u*] ← *time* # finished



finish `DFS-VISIT(G, b)`
time = 9
return to For-Loop of `DFS-VISIT(G, c)`

Depth-First Search

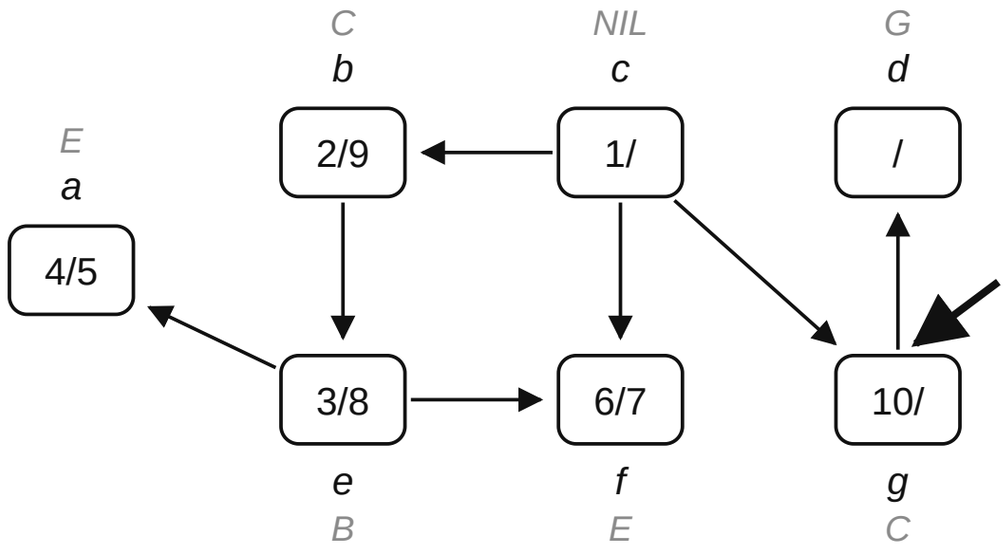
Procedure DFS-VISIT(*G*, *u*)
Inputs:
G: a graph
u: the vertex to be processed
Result: *u* and every vertex reachable from it are visited
time ← *time* + 1
d[*u*] ← *time* # discovered
color[*u*] ← GRAY
For each vertex *v* in neighbors(*u*):
    If color[*v*] = WHITE
        then π[*v*] ← *u*
            DFS-VISIT(*G*, *v*)
color[*u*] ← BLACK
time ← *time* + 1
f[*u*] ← *time* # finished



return to For-Loop of `DFS-VISIT(G, c)`
time = 9
Back in the For-Loop:
next adjacent white node: g
add parent of g
call `DFS-VISIT(G, g)`

Depth-First Search

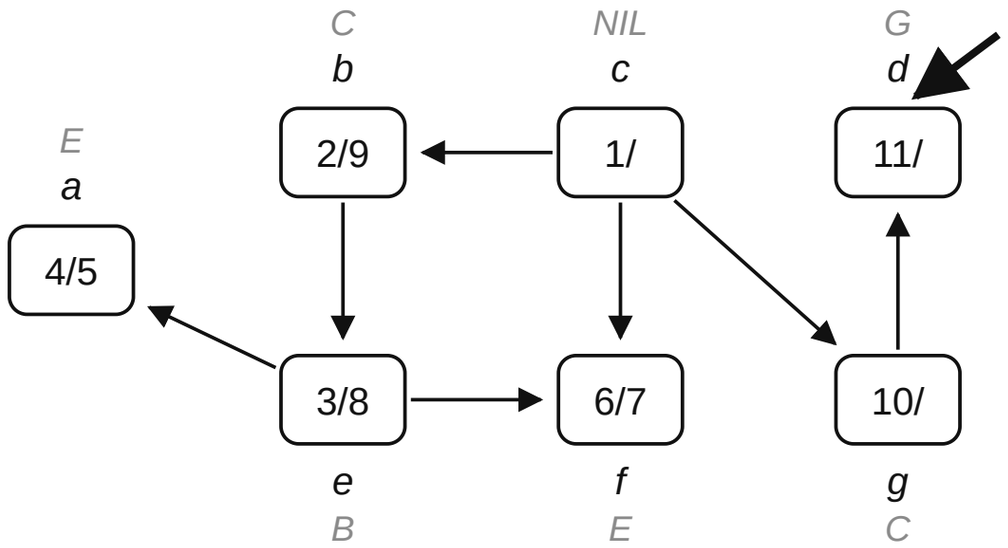
Procedure DFS-VISIT(*G*, *u*)
Inputs:
G: a graph
u: the vertex to be processed
Result: *u* and every vertex reachable from it are visited
time ← *time* + 1
d[*u*] ← *time* # discovered
color[*u*] ← GRAY
For each vertex *v* in neighbors(*u*):
    If color[*v*] = WHITE
        then π[*v*] ← *u*
            DFS-VISIT(*G*, *v*)
color[*u*] ← BLACK
time ← *time* + 1
f[*u*] ← *time* # finished



call `DFS-VISIT(G, g)`
time = 10
Enter the For-Loop:
next adjacent white node: d
add parent of d
call `DFS-VISIT(G, d)`

Depth-First Search

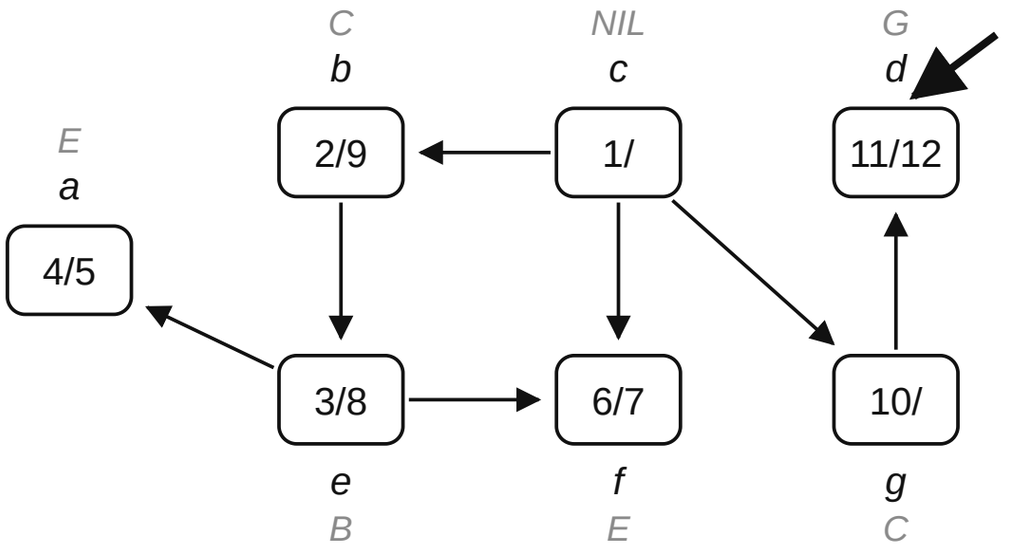
Procedure DFS-VISIT(*G*, *u*)
Inputs:
G: a graph
u: the vertex to be processed
Result: *u* and every vertex reachable from it are visited
time ← *time* + 1
d[*u*] ← *time* # discovered
color[*u*] ← GRAY
For each vertex *v* in neighbors(*u*):
    If color[*v*] = WHITE
        then π[*v*] ← *u*
            DFS-VISIT(*G*, *v*)
color[*u*] ← BLACK
time ← *time* + 1
f[*u*] ← *time* # finished



call `DFS-VISIT(G, d)`
time = 11
Enter the For-Loop:
no adjacent white nodes
finish `DFS-VISIT(G, d)`

Depth-First Search

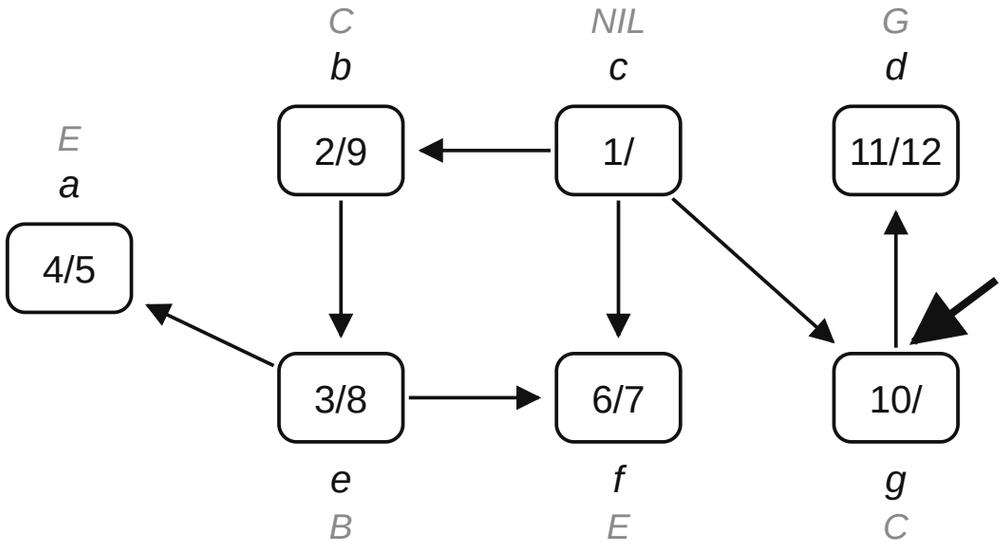
Procedure DFS-VISIT(*G*, *u*)
Inputs:
G: a graph
u: the vertex to be processed
Result: *u* and every vertex reachable from it are visited
time ← *time* + 1
d[*u*] ← *time* # discovered
color[*u*] ← GRAY
For each vertex *v* in neighbors(*u*):
    If color[*v*] = WHITE
        then π[*v*] ← *u*
            DFS-VISIT(*G*, *v*)
color[*u*] ← BLACK
time ← *time* + 1
f[*u*] ← *time* # finished



finish `DFS-VISIT(G, d)`
time = 12
return to For-Loop of `DFS-VISIT(G, g)`

Depth-First Search

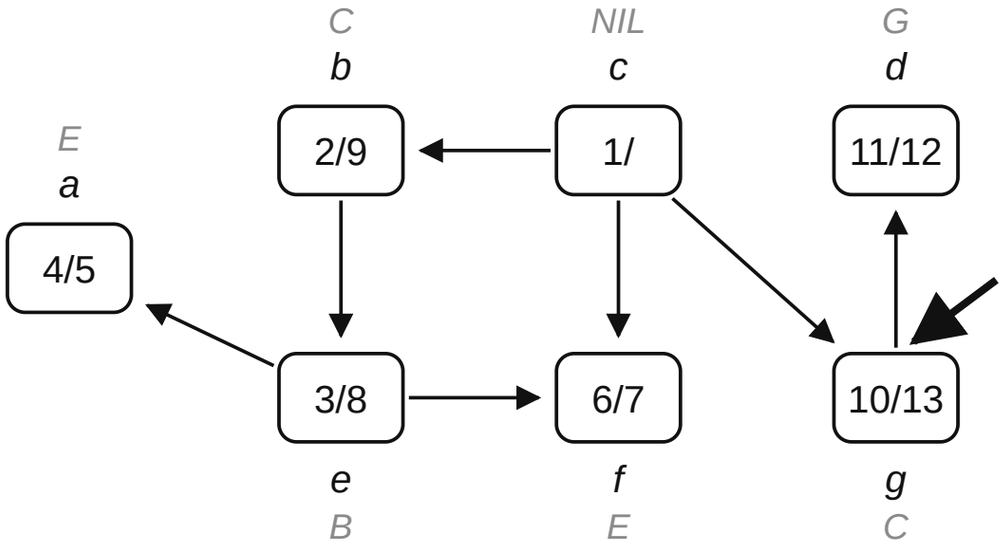
Procedure DFS-VISIT(*G*, *u*)
Inputs:
G: a graph
u: the vertex to be processed
Result: *u* and every vertex reachable from it are visited
time ← *time* + 1
d[*u*] ← *time* # discovered
color[*u*] ← GRAY
For each vertex *v* in neighbors(*u*):
    If color[*v*] = WHITE
        then π[*v*] ← *u*
            DFS-VISIT(*G*, *v*)
color[*u*] ← BLACK
time ← *time* + 1
f[*u*] ← *time* # finished



return to For-Loop of `DFS-VISIT(G, g)`
time = 12
Back in the For-Loop:
no adjacent white nodes
finish `DFS-VISIT(G, g)`

Depth-First Search

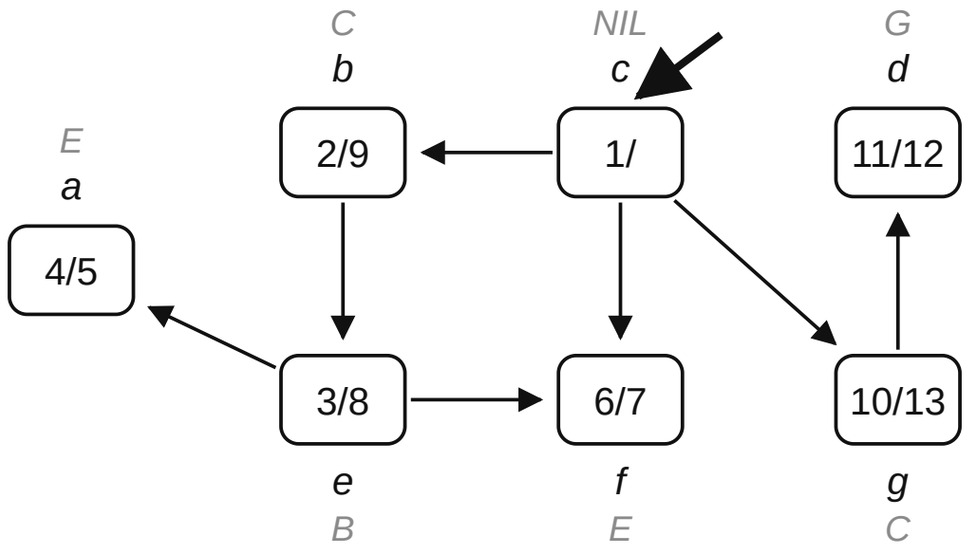
Procedure DFS-VISIT(*G*, *u*)
Inputs:
G: a graph
u: the vertex to be processed
Result: *u* and every vertex reachable from it are visited
time ← *time* + 1
d[*u*] ← *time* # discovered
color[*u*] ← GRAY
For each vertex *v* in neighbors(*u*):
    If color[*v*] = WHITE
        then π[*v*] ← *u*
            DFS-VISIT(*G*, *v*)
color[*u*] ← BLACK
time ← *time* + 1
f[*u*] ← *time* # finished



finish `DFS-VISIT(G, g)`
time = 13
return to For-Loop of `DFS-VISIT(G, c)`

Depth-First Search

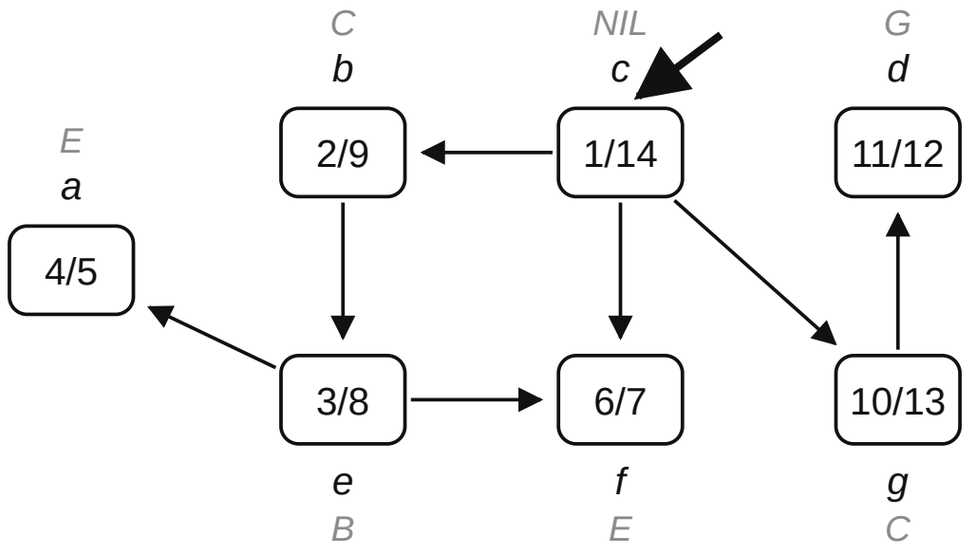
Procedure DFS-VISIT(*G*, *u*)
Inputs:
G: a graph
u: the vertex to be processed
Result: *u* and every vertex reachable from it are visited
time ← *time* + 1
d[*u*] ← *time* # discovered
color[*u*] ← GRAY
For each vertex *v* in neighbors(*u*):
    If color[*v*] = WHITE
        then π[*v*] ← *u*
            DFS-VISIT(*G*, *v*)
color[*u*] ← BLACK
time ← *time* + 1
f[*u*] ← *time* # finished



return to For-Loop of `DFS-VISIT(G, c)`
time = 13
Back in the For-Loop:
no adjacent white nodes
finish `DFS-VISIT(G, c)`

Depth-First Search

Procedure DFS-VISIT(*G*, *u*)
Inputs:
G: a graph
u: the vertex to be processed
Result: *u* and every vertex reachable from it are visited
time ← *time* + 1
d[*u*] ← *time* # discovered
color[*u*] ← GRAY
For each vertex *v* in neighbors(*u*):
    If color[*v*] = WHITE
        then π[*v*] ← *u*
            DFS-VISIT(*G*, *v*)
color[*u*] ← BLACK
time ← *time* + 1
f[*u*] ← *time* # finished



finish `DFS-VISIT(G, c)`
time = 14
All nodes finished
→ **FINISHED**

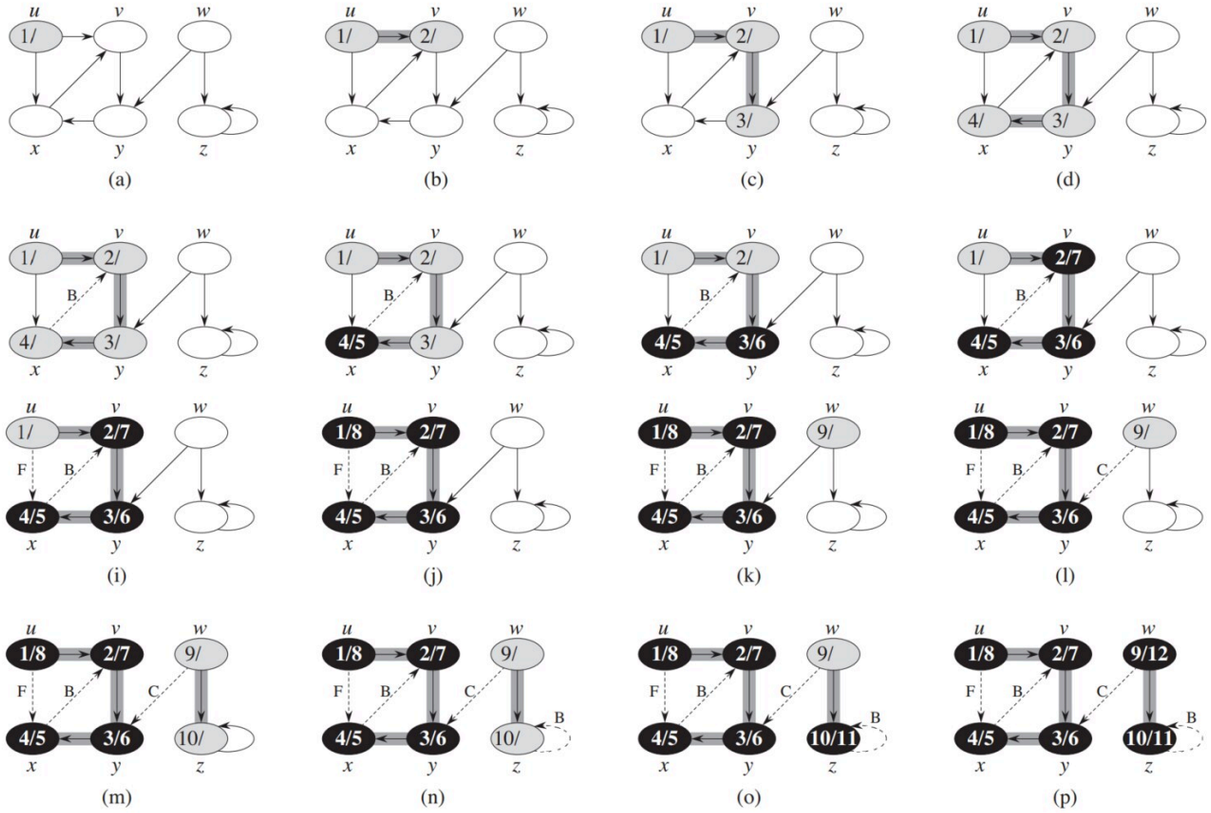
Depth-First Search

Universität Konstanz

More on Depth-First Search: See Cormen (2009)

```
DFS(G)
1  for each vertex u ∈ G.V
2      u.color = WHITE
3      u.π = NIL
4  time = 0
5  for each vertex u ∈ G.V
6      if u.color == WHITE
7          DFS-VISIT(G, u)

DFS-VISIT(G, u)
1  time = time + 1          // white vertex u has just been discovered
2  u.d = time
3  u.color = GRAY
4  for each v ∈ G.Adj[u]    // explore edge (u, v)
5      if v.color == WHITE
6          v.π = u
7          DFS-VISIT(G, v)
8  u.color = BLACK          // blacken u; it is finished
9  time = time + 1
10 u.f = time
```



From *Introduction to Algorithms* by Thomas Cormen (2009)

Types of recursive algorithms

- **linear** recursion
  - method calls itself once recursively
    e.g., factorial, gcd
- **tree** or **cascade** recursion
  - method calls itself multiple times recursively
    e.g., DFS, fibonacci numbers (next section)
- **tail-end** recursion
  - return value of a function is calculated as a call to itself and nothing else
    - gcd is tail-recursive
  - result of the last recursive call is the final return value
- other types: **nested** recursion
  - parameters of the method contain recursive calls
    e.g., Ackermann function:

$$ack(0,m) = m + 1$$

$$ack(n+1,0) = ack(n,1)$$

$$ack(n+1,m+1) = ack(n, ack(n+1,m))$$

Outline

1. Recursion
2. Recursive Algorithms
3. **Dynamic Programming**

Optimization Problems (Motivation)

- An **optimization problem** consists of maximizing or minimizing a real function by systematically choosing input values from within an allowed set and computing the value of the function
- finding “best available” values of some objective function given a defined domain or input
- many problems in applied social science research actually have this structure
  - optimizing a statistical model
  - optimizing the parameters of a simulation model
  - calculating the shortest geographical path

Dynamic Programming

Universität Konstanz

Definition of dynamic programming

Dynamic programming refers to simplifying a complicated problem by breaking it down into simpler sub-problems and **saving the result** for the future so that we will not have to compute that same problem again.

While some problems cannot be taken apart this way, problems that span several points in time often do break apart recursively.

- both a mathematical optimization method and an algorithm design principle
- common technique used in engineering and economics; goes back to the idea of Richard Bellman in the 1950s
- **optimal substructure**: if a problem can be solved optimally by breaking it into sub-problems and recursively finding optimal solutions to sub-problems

Dynamic Programming for Recursion

- Idea behind dynamic programming
  - use recursion but also save and re-use intermediate solutions:
    - breaking down an optimization problem into simpler sub-problems, and storing the solution to each sub-problem (called **memoization**) so that each sub-problem is only solved once
  - step-by-step put overlapping partial solutions together to form the overall solutions
  - requires checking global values at intermediate steps using all partial solutions
  - typical implementation through iteratively “filling up” an array or a matrix
  - makes code faster by reducing time complexity (reduced compute cycles) by efficient use of space

Example: Fibonacci numbers

Procedure FIB(n) # recursive version

Inputs:

n: a natural number ($n \in \mathbb{N}$)

Result: the n-th Fibonacci number

If $n = 0$ **then return** 0

If $n = 1$ **then return** 1

return FIB(n - 1) + FIB(n - 2)

The number of calls to `fib(n)` grows **exponentially** with n

Example: Fibonacci numbers

Procedure FIB(n) # recursive, with memoization

Inputs:

n: a natural number ($n \in \mathbb{N}$)

Data: memo: an array indexed from 0 to n, initially all NIL

Result: the n-th Fibonacci number

If $n = 0$ **then return** 0

If $n = 1$ **then return** 1

If *memo*[n] $\neq$ NIL **then return** *memo*[n]

memo[n] $\leftarrow$ FIB(n - 1) + FIB(n - 2)

return *memo*[n]

Example: Fibonacci numbers

Universität Konstanz

Procedure FIB(n) # recursive, with memoization

Inputs:

n: a natural number ($n \in \mathbb{N}$)

Data: memo: an array indexed from 0 to n, initially all NIL

Result: the n-th Fibonacci number

If $n = 0$ **then return** 0

If $n = 1$ **then return** 1

If $memo[n] \neq$ NIL **then return** $memo[n]$

$memo[n] \leftarrow$ FIB(n - 1) + FIB(n - 2)

return $memo[n]$

Previously calculated results are saved in `memo`

Example: Fibonacci numbers

Procedure FIB(n) # fully iterative

Inputs:

n: a natural number ($n \in \mathbb{N}$)

Data: *memo*: an array indexed from 0 to n

Result: the n-th Fibonacci number

memo[0] ← 0

memo[1] ← 1

For i = 2 to n:

    memo[i] ← *memo*[i - 1] + *memo*[i - 2]

return *memo*[n]

Outtake – Types of algorithms

Universität Konstanz

The most prevalent problem-solving heuristics you will encounter

- **Recursion**
  - optimization through recursive function call, e.g. DFS
  - requires specifying sensible abort conditions
- **Greedy**
  - successive extension of partial solutions, e.g. building a Huffman code
  - determines locally optimal choice at each step, then moves on
  - only correct if local optimum also leads to globally optimal solutions
- **Divide and Conquer**
  - separate the problem into decoupled/non-overlapping smaller sub-problems. More next session!
  - solve sub-problems independently
  - if necessary, collapse all sub-results to global results

Summary

- Recursion: a **base case** plus a **recursive case** that moves towards it
- Every call waits on the **program stack** until its own calls return
- **Factorial** and **Euclid's algorithm** run their recursive definitions
- **Depth-first search** records discovery and finishing times
- Recursion can be **linear**, **tree**, **tail-end** or **nested**
- **Dynamic programming** reuses the results of overlapping subproblems

Next session: Sorting Algorithms

Evaluation for Introduction to Programming with Python



https://evasys.uni-konstanz.de/evasys/online.php?pswd=DPQ9D